

Supervised Learning - Model Estimation

100 Questions: OLS, MLE, Gradient Descent, Regularization, Cross-Validation

GARP RAI Exam Preparation

What is Ordinary Least Squares (OLS) estimation in supervised learning?

ANSWER: B - A method that minimizes the sum of squared errors between predicted and actual values.

DETAILED EXPLANATION:

- OLS is the foundational estimation method in regression analysis
- Objective function: $\min \sum_{i=1}^N (y_i - \hat{y}_i)^2 = \min \sum_{i=1}^N \varepsilon_i^2$
- $\hat{y}_i = \beta_0 + \beta_1 x_{i1} + \dots + \beta_p x_{ip}$ is the predicted value
- $\varepsilon_i = y_i - \hat{y}_i$ is the residual (error)
- OLS minimizes the Residual Sum of Squares (RSS)
- Developed by Carl Friedrich Gauss in the 1790s
- Has a closed-form solution: $\hat{\beta} = (X^T X)^{-1} X^T y$

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes Least Absolute Deviations (LAD) - minimizes $\sum |\varepsilon_i|$ instead of $\sum \varepsilon_i^2$
- **C:** This describes Maximum Likelihood Estimation (MLE) - different paradigm entirely
- **D:** This describes regularization (L1/Lasso) - penalizes complexity

EXAM TRAP: OLS minimizes **squared errors**, NOT absolute errors! This is the most common confusion.

Which is the correct objective function for OLS?

ANSWER: B - $L = \sum_{i=1}^N (y_i - \hat{y}_i)^2$

DETAILED EXPLANATION:

- The OLS objective function is the Sum of Squared Errors (SSE)
- Also called Residual Sum of Squares (RSS) or Sum of Squared Residuals (SSR)
- Mathematical notation: $L(\beta) = \sum_{i=1}^N (y_i - X_i\beta)^2$
- This function is convex and differentiable $\Rightarrow$ has unique minimum
- First derivative: $\frac{\partial L}{\partial \beta} = -2X^T(y - X\beta)$
- Setting derivative to zero yields normal equations: $X^T X\beta = X^T y$

WHY OTHER OPTIONS ARE WRONG:

- **A:** L1 loss (Least Absolute Deviations) - not differentiable at zero
- **C:** L4 loss - rarely used, not OLS
- **D:** Chebyshev loss (minimax) - minimizes maximum error

EXAM TRAP: OLS = **RSS** = $\sum (y_i - \hat{y}_i)^2$! Know all three names (SSE, RSS, SSR).

Key difference between OLS and Nonlinear Least Squares?

ANSWER: B - OLS is used when model is linear in parameters; NLS when nonlinear in parameters.

DETAILED EXPLANATION:

- **Linear in parameters:** Model can be written as $\beta_0 + \beta_1 f_1(x) + \beta_2 f_2(x) + \dots$
 - Example: $y = \beta_0 + \beta_1 x + \beta_2 x^2$ (polynomial regression)
 - Even though nonlinear in x , it's linear in β s
- **Nonlinear in parameters:** Parameters appear nonlinearly
 - Example: $y = \beta_0 + \beta_1 e^{\beta_2 x}$
 - β_2 appears in exponent - nonlinear
- OLS has closed-form solution: $\hat{\beta} = (X^T X)^{-1} X^T y$
- NLS requires iterative numerical optimization

WHY OTHER OPTIONS ARE WRONG:

- **A:** Both minimize squared errors - this is the similarity, not difference
- **C:** Actually reversed - OLS has closed-form, NLS needs iterative methods
- **D:** Both are regression methods (continuous dependent variables)

EXAM TRAP: Difference is **linearity in parameters**, NOT in variables! $y = \beta_0 + \beta_1 x^2$ is OLS.

Which model requires Nonlinear Least Squares (NLS) estimation?

ANSWER: C - $y = \beta_0 + \beta_1 e^{\beta_2 x} + \varepsilon$

DETAILED EXPLANATION:

- Model C has β_2 in the exponent: $e^{\beta_2 x}$
- This is nonlinear in parameter β_2 - cannot be transformed to linear form
- Parameter β_1 multiplies the exponential term
- NLS requires iterative methods like Gauss-Newton or Levenberg-Marquardt
- The normal equations become nonlinear and cannot be solved directly

WHY OTHER OPTIONS ARE WRONG:

- **A:** $y = \beta_0 + \beta_1 x_1 + \beta_2 x_2$ - linear in all parameters (OLS)
- **B:** $y = \beta_0 + \beta_1 x + \beta_2 x^2$ - linear in β s (polynomial regression, OLS)
- **D:** $y = \beta_0 + \beta_1 \ln(x)$ - linear in β s (OLS with transformed x)

EXAM TRAP: NLS is about **nonlinearity in parameters**, not variables! This is the most common mistake.

What is Maximum Likelihood Estimation (MLE)?

ANSWER: B - Method that finds parameters maximizing probability (likelihood) of observing given data.

DETAILED EXPLANATION:

- MLE finds θ that maximizes $L(\theta) = P(\text{data}|\theta)$
- **Intuition:** "What parameters make the observed data most probable?"
- Likelihood function: $L(\theta) = \prod_{i=1}^N f(y_i|\theta)$ for i.i.d. data
- Log-likelihood: $\ell(\theta) = \sum_{i=1}^N \log f(y_i|\theta)$
- MLE is asymptotically consistent, efficient, and normally distributed
- Under regularity conditions, MLE achieves Cramér-Rao lower bound asymptotically

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes OLS - different estimation paradigm
- **C:** This describes regularization (penalizing complexity)
- **D:** Minimizing error variance is related to efficiency but not MLE definition

EXAM TRAP: MLE maximizes **likelihood** $P(\text{data}|\theta)$, NOT $P(\theta|\text{data})$!

Q6: OLS vs MLE Equivalence - Tutorial

When do OLS and MLE produce same parameter estimates?

ANSWER: B - When errors are normally distributed with constant variance.

DETAILED EXPLANATION:

- Model: $y_i = X_i\beta + \varepsilon_i$, with $\varepsilon_i \sim N(0, \sigma^2)$
- Likelihood: $L(\beta, \sigma^2) = \prod_{i=1}^N \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y_i - X_i\beta)^2}{2\sigma^2}\right)$
- Log-likelihood: $\ell(\beta) = -\frac{N}{2} \log(2\pi) - \frac{N}{2} \log(\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^N (y_i - X_i\beta)^2$
- Maximizing $\ell(\beta)$ is equivalent to minimizing $\sum (y_i - X_i\beta)^2$ (RSS)
- The σ^2 term doesn't affect the β estimates

WHY OTHER OPTIONS ARE WRONG:

- **A:** Independence alone insufficient - need normal distribution
- **C:** Uniform errors would give different estimator (MLE would minimize maximum absolute error)
- **D:** Autocorrelation violates OLS assumptions

EXAM TRAP: OLS and MLE are **equivalent** under normal errors! This is a critical exam concept.

What estimation method is typically used for logistic regression?

ANSWER: C - Maximum Likelihood Estimation (MLE)

DETAILED EXPLANATION:

- Logistic regression: $\log\left(\frac{p_i}{1-p_i}\right) = X_i\beta$
- $p_i = P(Y_i = 1|X_i) = \frac{1}{1+e^{-X_i\beta}}$
- Target is binary: $Y_i \in \{0, 1\}$
- Bernoulli likelihood: $L(\beta) = \prod_{i=1}^N p_i^{Y_i} (1 - p_i)^{1-Y_i}$
- Log-likelihood: $\ell(\beta) = \sum_{i=1}^N [Y_i \log(p_i) + (1 - Y_i) \log(1 - p_i)]$
- No closed-form solution - requires numerical optimization (Newton-Raphson, IRLS)

WHY OTHER OPTIONS ARE WRONG:

- **A:** OLS inappropriate - binary target, non-normal errors, heteroscedasticity
- **B:** NLS not standard - logistic regression's nonlinearity handled by MLE
- **D:** Method of Moments less common and less efficient

EXAM TRAP: Logistic regression uses **MLE**, NOT OLS! Critical distinction.

Q8: Likelihood Function - Tutorial

What does $L(\theta|\text{data})$ represent?

ANSWER: B - The probability of the data given the parameters.

DETAILED EXPLANATION:

- $L(\theta|\text{data}) = P(\text{data}|\theta)$
- **Key distinction:** Likelihood vs Posterior
 - Likelihood: $P(\text{data}|\theta)$ - frequentist paradigm
 - Posterior: $P(\theta|\text{data})$ - Bayesian paradigm
- Likelihood treats θ as fixed unknown constant
- Data is random; parameters are fixed
- Uses Bayes' theorem: $P(\theta|\text{data}) \propto P(\text{data}|\theta)P(\theta)$

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes posterior distribution $P(\theta|\text{data})$ - Bayesian concept
- **C:** This is the OLS loss function (RSS) - different concept
- **D:** This describes estimator variance - different statistical concept

EXAM TRAP: Likelihood = $P(\text{data}|\theta)$, NOT $P(\theta|\text{data})$! This confusion is extremely common.

Why is log-likelihood often used instead of likelihood?

ANSWER: B - Easier to maximize (products $\rightarrow$ sums, monotonic).

DETAILED EXPLANATION:

- **Mathematical convenience:**
 - Likelihood involves products: $L(\theta) = \prod_{i=1}^N f(y_i|\theta)$
 - Log-likelihood involves sums: $\ell(\theta) = \sum_{i=1}^N \log f(y_i|\theta)$
- **Monotonicity:** log is strictly increasing
 - Maximizing $\ell(\theta) \equiv$ maximizing $L(\theta)$
 - If $L(\theta_1) > L(\theta_2)$, then $\ell(\theta_1) > \ell(\theta_2)$
- **Numerical stability:**
 - Products of small probabilities can underflow to zero
 - Sums of logs are more numerically stable
- **Derivative simplification:** Sums easier to differentiate than products

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - log-likelihood is often negative (probabilities ≤ 1)
- **C:** False - usually requires numerical optimization
- **D:** False - doesn't affect parameter count (that's regularization)

EXAM TRAP: Log-likelihood is **monotonic** - maximizing it $\equiv$ maximizing likelihood!

Which estimation method typically has a closed-form solution?

ANSWER: C - Ordinary Least Squares for linear regression.

DETAILED EXPLANATION:

- **OLS closed-form:** $\hat{\beta} = (X^T X)^{-1} X^T y$
 - Derived from normal equations: $X^T X \hat{\beta} = X^T y$
 - Requires $X^T X$ to be invertible (no perfect multicollinearity)
 - Computational complexity: $O(p^2 N + p^3)$
- **Why others don't have closed-form:**
 - Logistic regression: nonlinear in parameters (link function)
 - NLS: nonlinear in parameters
 - Neural networks: highly complex composition of functions
- **When closed-form is not possible:**
 - When normal equations are nonlinear
 - When likelihood has no analytical solution
 - When loss function is not convex

WHY OTHER OPTIONS ARE WRONG:

- **A:** Logistic regression MLE requires numerical optimization (Newton-Raphson)
- **B:** NLS requires iterative numerical optimization
- **D:** Neural networks use backpropagation (numerical)

EXAM TRAP: Only **OLS** for **linear regression** has a closed-form solution!

Q11: OLS Assumptions - Tutorial

Which is NOT a standard OLS assumption?

ANSWER: D - The errors are autocorrelated.

DETAILED EXPLANATION:

- **Standard OLS assumptions (Gauss-Markov):**
 - 1 **Linearity in parameters:** $y = X\beta + \varepsilon$
 - 2 **Zero conditional mean:** $E[\varepsilon|X] = 0$
 - 3 **No perfect multicollinearity:** $X^T X$ invertible
 - 4 **Homoscedasticity:** $\text{Var}(\varepsilon|X) = \sigma^2$ (constant)
 - 5 **No autocorrelation:** $\text{Cov}(\varepsilon_i, \varepsilon_j|X) = 0$ for $i \neq j$
 - 6 **Normality:** $\varepsilon \sim N(0, \sigma^2)$ (for inference)
- **Autocorrelation** is a violation of the independence assumption
 - Common in time series data
 - Leads to inefficient estimates (not BLUE)
 - Standard errors are biased

WHY OTHER OPTIONS ARE WRONG:

- **A:** This IS an OLS assumption
- **B:** This IS an OLS assumption
- **C:** This IS an OLS assumption (for inference)

EXAM TRAP: OLS assumes **no autocorrelation** in errors! Autocorrelation is a violation.

Which is a desirable property of Maximum Likelihood Estimators?

ANSWER: B - They are consistent (converge to true values as sample size increases).

DETAILED EXPLANATION:

- **Consistency:** $\hat{\theta}_{MLE} \xrightarrow{P} \theta_0$ as $n \rightarrow \infty$
 - Under regularity conditions (identifiability, differentiability, etc.)
 - The estimator converges in probability to the true parameter
- **Asymptotic Normality:** $\sqrt{n}(\hat{\theta}_{MLE} - \theta_0) \xrightarrow{d} N(0, I^{-1}(\theta_0))$
 - $I(\theta)$ is the Fisher Information matrix
 - Achieves Cramér-Rao lower bound asymptotically
- **Asymptotic Efficiency:** Minimum variance among regular estimators
- **Invariance:** $\widehat{g(\theta)} = g(\hat{\theta})$ for continuous g

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - MLE can be biased (e.g., $\hat{\sigma}_{MLE}^2$ is biased for σ^2)
- **C:** False - asymptotically efficient, not always minimum variance in finite samples
- **D:** False - often requires numerical optimization

EXAM TRAP: MLE is **consistent** but NOT necessarily unbiased!

Why does NLS require iterative numerical optimization?

ANSWER: B - No closed-form solution for nonlinear-in-parameters models.

DETAILED EXPLANATION:

- **The problem:** Normal equations are nonlinear
 - OLS: $X^T X \beta = X^T y \Rightarrow$ linear system
 - NLS: $\frac{\partial S(\beta)}{\partial \beta} = 0$ where $S(\beta) = \sum (y_i - f(x_i, \beta))^2$
 - Partial derivatives $\frac{\partial S}{\partial \beta_j}$ are nonlinear functions of β
- **Common NLS algorithms:**
 - Gauss-Newton: Linearizes the model around current estimates
 - Levenberg-Marquardt: Combines Gauss-Newton and gradient descent
 - Gradient descent (first-order method)
 - Newton-Raphson (second-order method)
- **Convergence issues:**
 - Requires good starting values
 - May converge to local minima
 - May not converge at all if model is misspecified

WHY OTHER OPTIONS ARE WRONG:

- **A:** Multiple local minima are a concern but not the primary reason
- **C:** Error distribution doesn't determine need for numerical optimization
- **D:** Parameter count affects complexity but isn't fundamental

EXAM TRAP: No closed-form solution exists for **nonlinear-in-parameters** models!

Q14: Gauss-Markov Theorem - Tutorial

What does the Gauss-Markov theorem state about OLS estimators?

ANSWER: B - OLS estimators are the Best Linear Unbiased Estimators (BLUE) under certain conditions.

DETAILED EXPLANATION:

• **Gauss-Markov assumptions:**

- ① Linearity in parameters
- ② No perfect multicollinearity
- ③ Zero conditional mean: $E[\varepsilon|X] = 0$
- ④ Homoscedasticity: $\text{Var}(\varepsilon|X) = \sigma^2$
- ⑤ No autocorrelation: $\text{Cov}(\varepsilon_i, \varepsilon_j|X) = 0$

• **BLUE means:**

- **Best:** Minimum variance among all linear unbiased estimators
- **Linear:** Estimator is a linear function of y
- **Unbiased:** $E[\hat{\beta}] = \beta$
- **Estimator:** $\hat{\beta} = (X^T X)^{-1} X^T y$

• **Important:** OLS is BLUE among *linear* unbiased estimators

- Nonlinear estimators could have lower variance
- Biased estimators could have lower MSE (bias-variance tradeoff)

WHY OTHER OPTIONS ARE WRONG:

- **A:** Close but incomplete - misses "Best" (minimum variance)
- **C:** False - only among linear unbiased estimators
- **D:** False - consistency requires additional conditions

EXAM TRAP: OLS is **BLUE** under Gauss-Markov assumptions! Not "always best" among all estimators.

Q15: Estimation Methods Summary - Tutorial

Which estimation method for $y = \beta_0 + \beta_1 x_1^{\beta_2} + \varepsilon$?

ANSWER: D - Both NLS and MLE would be appropriate.

DETAILED EXPLANATION:

- **Model analysis:**

- $y = \beta_0 + \beta_1 x_1^{\beta_2} + \varepsilon$
- β_2 appears in the exponent of x_1
- This is nonlinear in parameter β_2
- Cannot be transformed to linear form (unlike x^2 which is linear in parameters)

- **NLS approach:**

- Minimizes $\sum (y_i - \beta_0 - \beta_1 x_{1i}^{\beta_2})^2$
- Requires numerical optimization
- Equivalent to MLE if errors are normal

- **MLE approach:**

- Defines likelihood: $L(\beta_0, \beta_1, \beta_2, \sigma^2)$
- If $\varepsilon \sim N(0, \sigma^2)$, MLE $\equiv$ NLS
- If errors not normal, MLE differs from NLS

WHY OTHER OPTIONS ARE WRONG:

- **A:** OLS requires linearity in parameters - this model violates that
- **B:** NLS alone correct but incomplete - MLE also works
- **C:** MLE alone correct but incomplete - NLS also works

EXAM TRAP: For nonlinear models, both **NLS** and **MLE** can be used!

Q16: Gradient Descent Definition - Tutorial

What is gradient descent in model estimation?

ANSWER: B - Iterative optimization algorithm updating parameters in direction of negative gradient.

DETAILED EXPLANATION:

- **Core concept:** Iteratively move in the direction of steepest descent
- **Update rule:** $\theta_{new} = \theta_{old} - \alpha \nabla L(\theta)$
- **Components:**
 - θ : Parameters being optimized
 - α : Learning rate (step size)
 - $\nabla L(\theta)$: Gradient (vector of partial derivatives)
- **Intuition:**
 - Gradient points in direction of steepest increase
 - Negative gradient points in direction of steepest decrease
 - Move in negative gradient direction to reduce loss
- **Convergence conditions:**
 - Loss function is differentiable
 - Learning rate is chosen appropriately
 - Function is convex for guaranteed global convergence

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - can get stuck in local minima
- **C:** This describes closed-form solutions
- **D:** This describes random search

EXAM TRAP: Gradient descent is **iterative** and uses **negative gradient**!

Q17: Gradient Descent Update Rule - Tutorial

What is the general update rule for gradient descent?

ANSWER: B - $\theta_{new} = \theta_{old} - \alpha \nabla L(\theta)$

DETAILED EXPLANATION:

- **Formula breakdown:**
 - θ_{new} : Updated parameter values
 - θ_{old} : Current parameter values
 - α : Learning rate (hyperparameter)
 - $\nabla L(\theta)$: Gradient of loss function
- **Why subtract?**
 - $\nabla L(\theta)$ points uphill (direction of increase)
 - $-\nabla L(\theta)$ points downhill (direction of decrease)
 - Subtracting moves toward lower loss
- **Example for linear regression:**
 - $L(\beta) = \sum (y_i - X_i \beta)^2$
 - $\nabla L(\beta) = -2X^T(y - X\beta)$
 - $\beta_{new} = \beta_{old} + 2\alpha X^T(y - X\beta)$

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is gradient ascent (moving to increase loss)
- **C:** Multiplication is not the correct operation
- **D:** Division is not the correct operation

EXAM TRAP: Gradient descent uses **subtraction** of the gradient!

Q18: Learning Rate Definition - Tutorial

What does the learning rate α control in gradient descent?

ANSWER: B - The step size of each parameter update.

DETAILED EXPLANATION:

• **Step size control:**

- α determines how far we move in the gradient direction
- Larger α : larger steps, faster potential convergence
- Smaller α : smaller steps, slower but more stable

• **Effects of learning rate:**

- **Too large:** Oscillation, divergence, failure to converge
- **Too small:** Very slow convergence, getting stuck in plateaus
- **Just right:** Efficient convergence to minimum

• **Common learning rate values:**

- Typical range: 10^{-6} to 10^{-1}
- Often use learning rate schedules (decay over time)
- Adaptive methods (Adam) adjust per parameter

• **Learning rate vs other hyperparameters:**

- Different from number of epochs (iterations)
- Different from batch size
- Different from regularization parameter λ

WHY OTHER OPTIONS ARE WRONG:

- **A:** Direction determined by gradient sign, not α
- **C:** Number of iterations is a different hyperparameter
- **D:** Initialization is a different step

EXAM TRAP: Learning rate controls **step size**, NOT direction!

Q19: Learning Rate Too Large - Tutorial

What happens if learning rate is set too large?

ANSWER: B - Algorithm may oscillate or diverge.

DETAILED EXPLANATION:

- **Consequences of large α :**
 - ① **Oscillation:** Parameter jumps back and forth across minimum
 - ② **Divergence:** Loss increases without bound
 - ③ **Instability:** Updates become erratic and unpredictable
- **Mathematical explanation:**
 - Consider quadratic: $f(\theta) = \frac{1}{2}\theta^2$
 - Gradient descent: $\theta_{t+1} = \theta_t - \alpha\theta_t = (1 - \alpha)\theta_t$
 - Convergence requires $|1 - \alpha| < 1 \Rightarrow 0 < \alpha < 2$
 - If $\alpha > 2$: divergence (oscillating with increasing magnitude)
 - If $\alpha = 2$: oscillation between θ and $-\theta$
- **Visual analogy:** Like walking down a hill with too large steps
 - Overshoot the bottom
 - End up higher than where you started
 - Keep oscillating or fall off the mountain

WHY OTHER OPTIONS ARE WRONG:

- **A:** Too small learning rate causes slow convergence
- **C:** Too large prevents convergence
- **D:** Algorithm already deterministic with fixed initialization

EXAM TRAP: Too large learning rate $\rightarrow$ **divergence or oscillation!**

Q20: Learning Rate Too Small - Tutorial

What happens if learning rate is set too small?

ANSWER: B - Algorithm converges very slowly and may get stuck.

DETAILED EXPLANATION:

- **Consequences of small α :**
 - 1 **Slow convergence:** Tiny steps require many iterations
 - 2 **Getting stuck:** May stop in plateaus or local minima
 - 3 **Wasted computation:** Many iterations with little progress
- **Mathematical explanation:**
 - For convex function: $\theta_{t+1} = \theta_t - \alpha \nabla f(\theta_t)$
 - If α is very small, θ_t changes very slowly
 - Convergence rate $O(1/t)$ for gradient descent
 - Small α increases the constant in $O(1/t)$
- **Visual analogy:** Like walking down a hill with tiny steps
 - Make very slow progress
 - May appear to stop on flat ground (plateau)
 - Will eventually reach bottom but takes forever
- **Diagnosis:**
 - Loss decreases very slowly
 - Requires many more epochs than expected
 - Often confused with convergence

WHY OTHER OPTIONS ARE WRONG:

- **A:** Too small gives slow convergence, not quick
- **C:** No algorithm finds global minimum instantly
- **D:** Too small is stable but slow

EXAM TRAP: Too small learning rate $\rightarrow$ **slow convergence and getting stuck!**

Q21: Batch Gradient Descent - Tutorial

What is Batch Gradient Descent?

ANSWER: B - Uses entire training dataset per update.

DETAILED EXPLANATION:

- **Definition:** Computes gradient using all N training samples
 - $\nabla L(\theta) = \frac{1}{N} \sum_{i=1}^N \nabla L_i(\theta)$
 - One update per epoch (full pass through data)
- **Characteristics:**
 - **Accurate gradient:** Uses all data, no sampling noise
 - **Stable:** Smooth convergence path
 - **Slow:** Each update requires $O(N)$ computation
 - **Memory intensive:** May need to store all data
- **Advantages:**
 - Guaranteed convergence for convex functions
 - No random noise in gradient estimates
 - Easier to analyze theoretically
- **Disadvantages:**
 - Impractical for large datasets ($N > 10^5$)
 - Redundant computations
 - Cannot escape local minima easily

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is SGD (Stochastic Gradient Descent)
- **C:** This is Mini-Batch Gradient Descent
- **D:** This is random search

EXAM TRAP: Batch GD uses **ALL data** for each update!

Q22: Stochastic Gradient Descent - Tutorial

What is Stochastic Gradient Descent (SGD)?

ANSWER: B - Uses a single randomly selected data point per iteration.

DETAILED EXPLANATION:

- **Definition:** Computes gradient using one randomly chosen sample
 - $\nabla L(\theta) \approx \nabla L_i(\theta)$ for random i
 - One update per data point (or per mini-batch)
 - Many updates per epoch
- **Characteristics:**
 - **Noisy gradient:** High variance in gradient estimates
 - **Fast:** Each update is $O(1)$ computation
 - **Escapes local minima:** Noise helps explore the landscape
 - **Converges to optimum:** With decreasing learning rate
- **Advantages:**
 - Scalable to huge datasets
 - Can escape shallow local minima
 - Online learning possible (streaming data)
- **Disadvantages:**
 - Noisy convergence (doesn't settle at exact minimum)
 - Requires careful learning rate scheduling
 - Harder to analyze theoretically

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is Batch GD (uses all data)
- **C:** This is Mini-Batch GD (uses subset)
- **D:** This describes learning rate scheduling

EXAM TRAP: SGD uses **ONE data point** per update!

Q23: Mini-Batch Gradient Descent - Tutorial

What is Mini-Batch Gradient Descent?

ANSWER: C - Uses a small random subset (batch) of data per update.

DETAILED EXPLANATION:

- **Definition:** Computes gradient using a small batch of size b
 - $\nabla L(\theta) \approx \frac{1}{b} \sum_{i \in \text{batch}} \nabla L_i(\theta)$
 - Typical batch sizes: 32, 64, 128, 256
 - One update per batch, multiple batches per epoch
- **Characteristics:**
 - **Balanced:** Compromise between Batch GD and SGD
 - **Less noisy than SGD:** Averaging reduces variance
 - **Faster than Batch GD:** Smaller computation per update
 - **Better than SGD:** More stable convergence
- **Advantages:**
 - Efficient for GPU processing (batch matrix operations)
 - Better gradient estimates than SGD
 - Faster convergence than Batch GD
 - Most common in deep learning practice
- **Batch size choice:**
 - Power of 2 for hardware efficiency (32, 64, 128)
 - Too small: noisy like SGD
 - Too large: slow like Batch GD

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is Batch GD (all data)
- **B:** This is SGD (one data point)
- **D:** This describes a stopping criterion

EXAM TRAP: Mini-Batch GD uses a **small random subset** of data!

Q24: SGD Advantage - Tutorial

Why is SGD often preferred over Batch GD for large datasets?

ANSWER: B - Computationally cheaper per iteration (fewer data points).

DETAILED EXPLANATION:

• **Computational cost comparison:**

- Batch GD: $O(N)$ per update (entire dataset)
- SGD: $O(1)$ per update (one data point)
- Mini-batch: $O(b)$ per update (batch size b)

• **Scaling to big data:**

- $N = 10^6$: Batch GD update requires 1M gradient computations
- SGD update requires 1 gradient computation
- With SGD, can make 1M updates in the time of one Batch update

• **SGD convergence properties:**

- May require more iterations to converge
- But each iteration is much cheaper
- Total time often much less than Batch GD

• **Additional benefits:**

- Can escape local minima due to noise
- Enables online learning
- Works with streaming data

WHY OTHER OPTIONS ARE WRONG:

- **A:** Not always - SGD converges noisily
- **C:** False - SGD doesn't guarantee global minimum
- **D:** False - SGD requires a loss function

EXAM TRAP: SGD preferred for large datasets due to **lower computational cost per iteration!**

Q25: Local Minima - Tutorial

What is a local minimum in gradient descent?

ANSWER: B - Gradient is zero but function value higher than at other nearby points.

DETAILED EXPLANATION:

- **Definition:**

- Point θ^* where $\nabla L(\theta^*) = 0$
- $L(\theta^*) < L(\theta)$ for all θ in neighborhood
- But not necessarily the lowest point overall

- **Mathematical characterization:**

- First-order condition: $\nabla L(\theta^*) = 0$
- Second-order condition: Hessian $H(\theta^*)$ is positive semidefinite
- All eigenvalues of H are ≥ 0

- **Why problematic:**

- Gradient descent stops (or slows) at local minima
- Cannot distinguish from global minimum
- May not find the best solution

- **Local vs Global minimum:**

- Local minimum: best in local neighborhood
- Global minimum: best overall
- Convex functions: every local minimum is global
- Non-convex functions: multiple local minima possible

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is global minimum (lowest overall)
- **C:** Gradient is zero at minima, not infinite
- **D:** This is a maximum (gradient zero, higher than neighbors)

EXAM TRAP: Local minima are **traps** where gradient descent can get stuck!

Q26: Saddle Points - Tutorial

What is a saddle point in gradient descent?

ANSWER: B - Gradient zero but curves upward in some directions and downward in others.

DETAILED EXPLANATION:

• **Definition:**

- Point θ^* where $\nabla L(\theta^*) = 0$
- Not a local minimum or maximum
- Hessian has both positive and negative eigenvalues

• **Mathematical characterization:**

- First-order: $\nabla L(\theta^*) = 0$
- Second-order: Hessian $H(\theta^*)$ is indefinite
- Some eigenvalues > 0 (upward curves)
- Some eigenvalues < 0 (downward curves)

• **Why important:**

- More common than local minima in high dimensions
- Gradient descent slows down near saddle points
- Requires momentum or noise to escape

• **Comparison to local minima:**

- Local minima: all eigenvalues positive
- Saddle points: mixed eigenvalues
- In high dimensions, saddle points dominate

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes a local minimum
- **C:** Gradient is zero at saddle points
- **D:** This is global minimum

EXAM TRAP: Saddle points have **zero gradient** but are neither minima nor maxima!

What is the purpose of momentum in gradient descent?

ANSWER: B - Accelerate convergence and reduce oscillations by accumulating past gradients.

DETAILED EXPLANATION:

- **Definition:** Momentum adds fraction of previous update to current update
 - $v_t = \beta v_{t-1} + (1 - \beta) \nabla L(\theta_t)$
 - $\theta_{t+1} = \theta_t - \alpha v_t$
 - β is momentum parameter (typically 0.9)
- **How it works:**
 - Accumulates gradient history
 - Accelerates in consistent gradient directions
 - Dampens oscillations in perpendicular directions
- **Benefits:**
 - **Faster convergence:** Smooths out zigzagging
 - **Better local minima:** Can roll past shallow minima
 - **Escape saddle points:** Built-up momentum pushes through
- **Analogy:** Ball rolling down a hill
 - Gains speed as it rolls
 - Overcomes small bumps
 - Doesn't stop at first flat spot

WHY OTHER OPTIONS ARE WRONG:

- **A:** Learning rate decay is separate concept
- **C:** Batch size is separate concept
- **D:** Initialization is separate step

EXAM TRAP: Momentum uses **past gradients** to smooth updates!

Q28: Adam Optimizer - Tutorial

What is the Adam optimizer?

ANSWER: B - Adaptive learning rate optimizer combining momentum and RMSprop.

DETAILED EXPLANATION:

- **Definition:** Adaptive Moment Estimation
 - Combines advantages of two methods
 - Momentum: tracks first moment (mean of gradients)
 - RMSprop: tracks second moment (variance of gradients)
- **Update formulas:**
 - $m_t = \beta_1 m_{t-1} + (1 - \beta_1) \nabla L(\theta_t)$
 - $v_t = \beta_2 v_{t-1} + (1 - \beta_2) (\nabla L(\theta_t))^2$
 - $\hat{m}_t = m_t / (1 - \beta_1^t)$ (bias correction)
 - $\hat{v}_t = v_t / (1 - \beta_2^t)$ (bias correction)
 - $\theta_{t+1} = \theta_t - \alpha \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$
- **Key features:**
 - Adaptive learning rate per parameter
 - Handles sparse gradients well
 - Requires minimal tuning
 - Most popular optimizer in deep learning
- **Default hyperparameters:**
 - $\alpha = 0.001$ (learning rate)
 - $\beta_1 = 0.9$ (momentum decay)
 - $\beta_2 = 0.999$ (RMSprop decay)
 - $\epsilon = 10^{-8}$ (small constant for stability)

WHY OTHER OPTIONS ARE WRONG:

- **A:** Adam adapts learning rate dynamically
- **C:** Adam uses first AND second moments
- **D:** Adam is an optimizer, not initialization

EXAM TRAP: Adam is **adaptive** - adjusts learning rates per parameter!

Q29: Learning Rate Scheduling - Tutorial

What is learning rate scheduling?

ANSWER: B - Adjusts learning rate during training according to predefined schedule.

DETAILED EXPLANATION:

- **Definition:** Systematic reduction of learning rate over time
 - Larger learning rates initially (exploration)
 - Smaller learning rates later (fine-tuning)
- **Why schedule is needed:**
 - Large LR: make progress quickly, explore the space
 - Small LR: settle into minimum, avoid oscillation
 - Both are needed but at different times
- **Common schedules:**
 - 1 **Step decay:** Reduce by factor at fixed intervals
 - 2 **Exponential decay:** $\alpha_t = \alpha_0 \times \gamma^t$
 - 3 **Cosine annealing:** Smooth cyclical variation
 - 4 **Warmup:** Increase then decrease
- **Practical advice:**
 - Start with larger learning rate
 - Reduce when validation loss plateaus
 - Different schedules work for different problems

WHY OTHER OPTIONS ARE WRONG:

- **A:** Random changes are not a "schedule"
- **C:** Constant LR is the opposite of scheduling
- **D:** Batch size is a different hyperparameter

EXAM TRAP: Learning rate schedules **adjust learning rate over time!**

Q30: Step Decay - Tutorial

What is step decay in learning rate scheduling?

ANSWER: B - Learning rate decreases by factor at specific intervals.

DETAILED EXPLANATION:

- **Definition:** Piecewise constant reduction
 - $\alpha_t = \alpha_0 \times \gamma^{\lfloor t/T \rfloor}$
 - γ : decay factor (e.g., 0.1 or 0.5)
 - T : interval size (e.g., every 10 epochs)
- **Example:**
 - $\alpha_0 = 0.01, \gamma = 0.1, T = 10$
 - Epochs 0-9: $\alpha = 0.01$
 - Epochs 10-19: $\alpha = 0.001$
 - Epochs 20-29: $\alpha = 0.0001$
- **Advantages:**
 - Simple and easy to implement
 - Predictable behavior
 - Works well for many problems
- **Disadvantages:**
 - Abrupt changes in learning rate
 - Requires choosing interval and factor
 - Not adaptive to data
- **When to use:**
 - When training loss plateaus
 - When model is stable
 - Common in practice

WHY OTHER OPTIONS ARE WRONG:

- **A:** Continuous decrease is exponential decay
- **C:** Increasing LR is a different strategy
- **D:** Random changes are not step decay

EXAM TRAP: Step decay reduces learning rate **at specific intervals!**

Q31: Exponential Decay - Tutorial

What is exponential decay in learning rate scheduling?

ANSWER: A - $\alpha_t = \alpha_0 \times \gamma^t$

DETAILED EXPLANATION:

- **Definition:** Continuous multiplicative decay
 - $\alpha_t = \alpha_0 \times \gamma^t$
 - $\gamma \in (0, 1)$: decay rate
 - α_0 : initial learning rate
- **Example:**
 - $\alpha_0 = 0.01, \gamma = 0.95$
 - Epoch 0: $\alpha = 0.01$
 - Epoch 10: $\alpha = 0.01 \times 0.95^{10} \approx 0.006$
 - Epoch 100: $\alpha = 0.01 \times 0.95^{100} \approx 0.00006$
- **Properties:**
 - **Smooth decay:** No abrupt changes
 - **Consistent:** Same relative decrease each step
 - **Long tail:** Never reaches zero (asymptotically)
- **Choosing γ :**
 - $\gamma = 0.9$: faster decay
 - $\gamma = 0.99$: slower decay
 - $\gamma = 0.95$: common default
- **Comparison to step decay:**
 - Step decay: discrete jumps
 - Exponential decay: smooth continuous decrease

WHY OTHER OPTIONS ARE WRONG:

- **B:** This is inverse time decay ($\alpha_t = \alpha_0/t$)
- **C:** This is linear decay
- **D:** This is constant learning rate

EXAM TRAP: Exponential decay: $\alpha_t = \alpha_0 \times \gamma^t$!

Q32: Convergence Definition - Tutorial

What does convergence mean in gradient descent?

ANSWER: C - Loss has stopped decreasing significantly (or gradient near zero).

DETAILED EXPLANATION:

- **Definition:** Algorithm has reached a stationary point
 - Gradient norm $\|\nabla L(\theta)\| < \varepsilon$
 - Loss change $\Delta L < \delta$
 - Parameter change $\|\theta_t - \theta_{t-1}\| < \tau$
- **Convergence criteria:**
 - 1 **Gradient norm:** $\|\nabla L(\theta_t)\| < 10^{-5}$
 - 2 **Loss change:** $|L(\theta_t) - L(\theta_{t-1})| < 10^{-6}$
 - 3 **Parameter change:** $\|\theta_t - \theta_{t-1}\| < 10^{-5}$
 - 4 **Validation loss:** Stopping when validation loss increases
- **Convergence vs global minimum:**
 - Convergence $\neq$ global minimum
 - Could be local minimum or saddle point
 - Function is convex: convergence $\Rightarrow$ global minimum
- **Stopping vs convergence:**
 - Max iterations: stopping criterion, not convergence
 - Convergence: based on progress, not iteration count

WHY OTHER OPTIONS ARE WRONG:

- **A:** Ideal case but not guaranteed
- **B:** Could be due to zero learning rate
- **D:** This is a stopping criterion, not convergence

EXAM TRAP: Convergence means **loss has stopped decreasing** significantly!

Q33: Loss Landscape - Tutorial

In high-dimensional spaces, which is more common: local minima or saddle points?

ANSWER: B - Saddle points are more common.

DETAILED EXPLANATION:

- **Theoretical result:** Saddle points dominate in high dimensions
 - For a random function in d dimensions
 - Probability of a critical point being local minimum: 2^{-d}
 - Probability of being saddle point: $1 - 2^{-d}$
 - For $d = 100$: $P(\text{local min}) \approx 10^{-30}$
- **Intuition:**
 - Local minimum requires all directions to be upward
 - Saddle point has some up and some down directions
 - More likely to have mixed directions in high dimensions
- **Implications for deep learning:**
 - Local minima are not the main problem
 - Saddle points are more common
 - Gradient descent can escape saddle points
 - Momentum and noise help escape
- **Recent research findings:**
 - Deep networks have many saddle points
 - Most local minima are nearly global
 - The loss landscape is surprisingly well-behaved

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - local minima are rare in high dimensions
- **C:** False - no guarantee of global minimum
- **D:** False - both exist but saddle points more common

EXAM TRAP: Saddle points are **more common** than local minima in high dimensions!



Q34: Vanishing Gradients - Tutorial

What is the vanishing gradient problem?

ANSWER: B - Gradients become very small during backpropagation, preventing early layers from learning.

DETAILED EXPLANATION:

- **Definition:** Gradients shrink exponentially as they propagate backward
 - Early layers receive very small gradients
 - Weights in early layers barely update
 - Network stops learning effectively
- **Causes:**
 - Activation functions with small derivatives
 - **Sigmoid:** derivative $\sigma(x)(1 - \sigma(x)) \leq 0.25$
 - **tanh:** derivative $\text{sech}^2(x) \rightarrow 0$ for large $|x|$
 - Deep networks (many layers multiply small values)
- **Signs of vanishing gradients:**
 - Early layers have very small gradient norms
 - Training loss doesn't decrease
 - Early layers look like random initialization
- **Solutions:**
 - ReLU activation (derivative 1 for positive inputs)
 - Residual connections (skip connections)
 - Batch normalization
 - Careful weight initialization (Xavier, He)

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes exploding gradients
- **C:** This describes oscillations
- **D:** Extreme case, not the general problem

EXAM TRAP: Vanishing gradients = **very small gradients** in early layers!

Q35: Exploding Gradients - Tutorial

What is the exploding gradient problem?

ANSWER: B - Gradients become very large during backpropagation, causing unstable updates.

DETAILED EXPLANATION:

- **Definition:** Gradients grow exponentially as they propagate backward
 - Large gradients lead to large parameter updates
 - Parameters become very large
 - Training becomes unstable or diverges
- **Causes:**
 - Large initial weights
 - Poor weight initialization
 - Long sequences (RNNs)
 - Repeated multiplication of values > 1
- **Signs of exploding gradients:**
 - Gradients become extremely large ($> 10^6$)
 - Loss becomes NaN (not a number)
 - Parameters become extremely large
 - Training diverges (loss increases)
- **Solutions:**
 - **Gradient clipping:** Cap gradients at threshold
 - Norm clipping: $\nabla L \leftarrow \frac{\nabla L}{\|\nabla L\|} \times \text{threshold}$
 - Value clipping: $\nabla L \leftarrow \min(\max(\nabla L, -\text{threshold}), \text{threshold})$
 - Careful weight initialization
 - Smaller learning rates
 - L2 regularization (weight decay)

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes vanishing gradients
- **C:** This is opposite (zero gradients)
- **D:** This describes a different phenomenon

EXAM TRAP: Exploding gradients = **very large gradients** causing instability!

Q36: Backpropagation Definition - Tutorial

What is backpropagation in neural networks?

ANSWER: B - Algorithm that computes gradients of loss with respect to all weights using chain rule.

DETAILED EXPLANATION:

- **Definition:** Efficient algorithm for computing gradients
 - Uses chain rule of calculus
 - Propagates errors backward from output to input
 - Computes $\frac{\partial L}{\partial w_i}$ for every weight
- **Algorithm steps:**
 - 1 **Forward pass:** Compute predictions and loss
 - 2 **Backward pass:** Compute gradients of loss
 - 3 **Update:** Use gradients to update weights
- **Key insight:**
 - Reuses computations efficiently
 - Each gradient computed once
 - $O(N)$ complexity for N parameters
- **Historical context:**
 - Invented independently multiple times
 - Popularized by Rumelhart, Hinton, Williams (1986)
 - Made deep learning possible

WHY OTHER OPTIONS ARE WRONG:

- **A:** Initialization is a separate step
- **C:** Architecture selection is separate
- **D:** Activation function selection is separate

EXAM TRAP: Backpropagation uses **chain rule** to compute gradients!

Q37: Chain Rule - Tutorial

Which mathematical rule is the foundation of backpropagation?

ANSWER: C - The chain rule.

DETAILED EXPLANATION:

• **Chain rule in calculus:**

- For composite functions: $\frac{d}{dx} f(g(x)) = f'(g(x))g'(x)$
- Neural networks are compositions of many functions
- Loss $L = f_N(f_{N-1}(\dots f_1(x)\dots))$

• **Backpropagation application:**

- $\frac{\partial L}{\partial w_{ij}^{(l)}} = \frac{\partial L}{\partial z_i^{(l+1)}} \cdot \frac{\partial z_i^{(l+1)}}{\partial z_j^{(l)}} \cdot \dots$
- Each layer contributes one factor
- Gradients are multiplied backward

• **Example:**

- $z = w_2 \cdot \sigma(w_1 x)$
- $\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial a} \cdot \frac{\partial a}{\partial w_1}$
- $a = \sigma(w_1 x)$

• **Why not other rules:**

- Product rule: for products, not compositions
- Quotient rule: for ratios
- Power rule: for powers

WHY OTHER OPTIONS ARE WRONG:

- **A:** Product rule applies to products of functions
- **B:** Quotient rule applies to ratios
- **D:** Power rule applies to powers

EXAM TRAP: Backpropagation = **chain rule** applied to neural networks!

Q38: Forward Pass - Tutorial

What happens during forward pass in backpropagation?

ANSWER: B - Input data passed through network to compute output and loss.

DETAILED EXPLANATION:

• **Forward pass steps:**

- ① Input data enters the network
- ② Passed through hidden layers
- ③ Each layer applies: $z^{(l)} = W^{(l)} a^{(l-1)} + b^{(l)}$
- ④ Activation: $a^{(l)} = \sigma(z^{(l)})$
- ⑤ Output layer produces predictions $\hat{y}$
- ⑥ Loss L is computed: $L = \text{Loss}(y, \hat{y})$

• **What is stored:**

- Inputs to each layer $z^{(l)}$
- Activations $a^{(l)}$
- Loss value
- Used in backward pass for gradient computation

• **Complexity:**

- One forward pass per batch
- $O(N)$ operations where N = number of parameters
- Must be computed before backward pass

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is backward pass (gradients and updates)
- **C:** Optimization happens over many iterations
- **D:** Initialization happens before training

EXAM TRAP: Forward pass computes **output and loss**; backward pass computes gradients!

What happens during backward pass in backpropagation?

ANSWER: B - Gradients of loss with respect to each weight computed using chain rule.

DETAILED EXPLANATION:

• **Backward pass steps:**

- ① Start from output layer
- ② Compute $\delta^{(L)} = \frac{\partial L}{\partial a^{(L)}} \odot \sigma'(z^{(L)})$
- ③ Propagate backward: $\delta^{(l)} = (W^{(l+1)})^T \delta^{(l+1)} \odot \sigma'(z^{(l)})$
- ④ Compute weight gradients: $\frac{\partial L}{\partial W^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$
- ⑤ Compute bias gradients: $\frac{\partial L}{\partial b^{(l)}} = \delta^{(l)}$

• **What is used:**

- Activations from forward pass
- Loss gradient
- Chain rule applied layer by layer

• **Complexity:**

- One backward pass per batch
- $O(N)$ operations
- Computes gradients for all parameters

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes forward pass
- **C:** Learning rate adjustment is separate
- **D:** Loss evaluation happens during forward pass

EXAM TRAP: Backward pass computes **gradients** of all weights!

Q40: Weight Update - Tutorial

After computing gradients, how are weights updated?

ANSWER: B - $w_{new} = w_{old} - \alpha \nabla L(w)$

DETAILED EXPLANATION:

- **Update rule:**

- $w_{new} = w_{old} - \alpha \nabla L(w)$
- α : learning rate
- $\nabla L(w)$: gradient computed by backpropagation

- **Why this form:**

- Gradient points uphill (direction of increasing loss)
- Negative gradient points downhill (decreasing loss)
- Move in direction of decreasing loss

- **Example for simple network:**

- $w_{new} = w_{old} - \alpha \frac{\partial L}{\partial w}$
- Applied to every weight and bias
- Update after each batch (or epoch)

- **Variations:**

- Momentum: include previous updates
- Adam: adaptive learning rates
- Weight decay: add regularization

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is gradient ascent (increases loss)
- **C:** Ignores learning rate and old weights
- **D:** Multiplication alone ignores gradient

EXAM TRAP: Weights updated by **subtracting** gradient $\times$ learning rate!

Q41: Computational Graph - Tutorial

What is a computational graph in backpropagation?

ANSWER: B - Graph representing forward computation as DAG enabling efficient gradient computation.

DETAILED EXPLANATION:

- **Definition:** Directed Acyclic Graph (DAG) of operations
 - Nodes: operations (add, multiply, activation)
 - Edges: data flow (tensors)
 - Represents the entire computation
- **Forward pass in graph:**
 - Input nodes $x_1, x_2, \dots$
 - Operation nodes: $+$, $\times$, σ , ...
 - Output: loss value
 - Traverse forward from inputs to outputs
- **Backward pass in graph:**
 - Start from loss node
 - Traverse backward using chain rule
 - Each node computes gradient of its output
 - Gradients propagate backward
- **Frameworks using this:**
 - TensorFlow (static graphs)
 - PyTorch (dynamic graphs)
 - JAX, Theano, etc.

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is architecture diagram
- **C:** This is data distribution
- **D:** This is loss function plot

EXAM TRAP: Computational graphs enable **automatic differentiation** and backpropagation!



Q42: Automatic Differentiation - Tutorial

What is automatic differentiation in deep learning frameworks?

ANSWER: B - Automatic computation of gradients using chain rule on computational graphs.

DETAILED EXPLANATION:

- **Definition:** "Autodiff" = automatic gradient computation
 - Uses computational graph
 - Applies chain rule automatically
 - Framework handles the details
- **How it works:**
 - 1 Build computational graph
 - 2 Forward pass computes outputs
 - 3 Backward pass computes gradients
 - 4 Returns gradients for all parameters
- **Modes of autodiff:**
 - **Forward mode:** Compute derivatives with respect to inputs
 - **Reverse mode:** Compute derivatives with respect to outputs (backpropagation)
 - Reverse mode is more efficient for many parameters
- **Advantages:**
 - No manual derivative derivation
 - Less error-prone
 - Works for complex architectures
 - Enables rapid prototyping

WHY OTHER OPTIONS ARE WRONG:

- **A:** Learning rate selection is separate
- **C:** Architecture selection is separate
- **D:** Data preprocessing is separate

EXAM TRAP: Automatic differentiation is how frameworks implement [backpropagation!](#)

Q43: Memory Requirements - Tutorial

What is a key memory requirement for backpropagation?

ANSWER: A - Requires storing all intermediate activations from forward pass.

DETAILED EXPLANATION:

- **Why activations must be stored:**
 - Backward pass needs them to compute gradients
 - $\frac{\partial L}{\partial w^{(l)}} = \delta^{(l)} (a^{(l-1)})^T$
 - $a^{(l-1)}$ comes from forward pass
- **Memory breakdown:**
 - Model parameters: $O(p)$ memory
 - Activations: $O(N \times \text{batch_size} \times \text{layers})$
 - Gradients: $O(p)$ memory
 - Total: often dominated by activations
- **Memory optimization techniques:**
 - Checkpointing: recompute activations when needed
 - Mixed precision: use lower precision storage
 - Gradient checkpointing: trade compute for memory
 - Smaller batch sizes
- **Why this matters:**
 - Limits batch size
 - Limits model size
 - Affects training speed

WHY OTHER OPTIONS ARE WRONG:

- **B:** False - activations are needed
- **C:** False - final loss alone is insufficient
- **D:** False - gradients are computed during backward pass

EXAM TRAP: Backpropagation requires storing **all intermediate activations!**

Q44: Training Loop Order - Tutorial

What is correct order of operations in training loop?

ANSWER: A - Forward pass → Compute loss → Backward pass → Update weights

DETAILED EXPLANATION:

• **Step-by-step breakdown:**

① **Forward pass:**

- Input data through network
- Compute predictions $\hat{y}$
- Store activations

② **Compute loss:**

- Compare $\hat{y}$ to actual y
- $L = \text{Loss}(y, \hat{y})$

③ **Backward pass:**

- Compute $\frac{\partial L}{\partial w}$ for all weights
- Use chain rule
- Propagate gradients backward

④ **Update weights:**

- $w_{new} = w_{old} - \alpha \nabla L(w)$
- Apply gradient descent step

• **Why this order:**

- Need predictions before loss
- Need loss before gradients
- Need gradients before updates

WHY OTHER OPTIONS ARE WRONG:

- **B:** Backward pass cannot happen before forward pass
- **C:** Loss cannot be computed before forward pass
- **D:** Updates cannot happen before gradients

EXAM TRAP: Forward → Loss → Backward → Update!

Q45: Backpropagation Summary - Tutorial

Which statement about backpropagation is **FALSE**?

ANSWER: D - Backpropagation always finds the global minimum of the loss function.

DETAILED EXPLANATION:

- **Why D is false:**
 - No guarantee of global minimum
 - Can get stuck in local minima
 - Can get trapped in saddle points
 - Only guaranteed for convex functions
 - Neural networks are non-convex
- **What backpropagation actually does:**
 - Computes gradients efficiently
 - Enables gradient descent
 - Finds a local minimum (often good enough)
 - Converges to stationary point
- **Why other statements are true:**
 - **A:** True - computes gradients for all weights
 - **B:** True - uses chain rule
 - **C:** True - requires storing activations

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is exactly what backpropagation does
- **B:** This is the mathematical foundation
- **C:** This is the memory requirement

EXAM TRAP: Backpropagation does **NOT** guarantee finding the global minimum!

What is the bias-variance tradeoff in supervised learning?

ANSWER: B - Tradeoff between model's ability to fit training data (bias) and generalize to new data (variance).

DETAILED EXPLANATION:

- **Mathematical decomposition:**

- $E[(Y - \hat{f}(X))^2] = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$
- **Bias²:** $[E[\hat{f}(X)] - f(X)]^2$
- **Variance:** $E[(\hat{f}(X) - E[\hat{f}(X)])^2]$
- **Irreducible:** σ^2 (noise in data)

- **Intuition:**

- **Bias:** How wrong the model is on average
- **Variance:** How much the model changes with different training data
- **Tradeoff:** Can't reduce both simultaneously

- **Model complexity relationship:**

- Simple models: High bias, Low variance
- Complex models: Low bias, High variance
- Optimal: Balance between the two

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is accuracy-interpretability tradeoff
- **C:** This is speed-accuracy tradeoff
- **D:** This is curse of dimensionality

EXAM TRAP: Bias = error from assumptions; Variance = error from sensitivity to training data!

Q47: Bias Definition - Tutorial

What does "bias" refer to in the bias-variance tradeoff?

ANSWER: A - Error introduced by the model's assumptions and limitations.

DETAILED EXPLANATION:

- **Definition:** Error due to model's simplifying assumptions
 - Assumption of linearity when relationship is nonlinear
 - Assumption of independence when data has structure
 - Limited capacity of the model
- **Mathematical definition:**
 - $\text{Bias} = E[\hat{f}(X)] - f(X)$
 - Expected difference between predicted and true function
 - Over multiple training sets
- **High bias characteristics:**
 - Underfitting
 - Poor performance on training data
 - Poor performance on test data
 - Simple model (linear, shallow)
- **Examples of high bias models:**
 - Linear regression for nonlinear data
 - Logistic regression for complex classification
 - Small decision trees

WHY OTHER OPTIONS ARE WRONG:

- **B:** This describes variance (sensitivity to training data)
- **C:** This describes irreducible error
- **D:** This describes optimization error

EXAM TRAP: Bias = error from model assumptions (underfitting)!

Q48: Variance Definition - Tutorial

What does "variance" refer to in the bias-variance tradeoff?

ANSWER: B - Error introduced by model's sensitivity to fluctuations in training data.

DETAILED EXPLANATION:

- **Definition:** Error due to sensitivity to training data
 - Different training sets produce different models
 - High variance models change a lot with data
 - Captures noise as well as signal
- **Mathematical definition:**
 - Variance = $E[(\hat{f}(X) - E[\hat{f}(X)])^2]$
 - Variability of predictions around expected prediction
 - Over multiple training sets
- **High variance characteristics:**
 - Overfitting
 - Excellent performance on training data
 - Poor performance on test data
 - Complex model (deep, many parameters)
- **Examples of high variance models:**
 - High-degree polynomial with too many terms
 - Deep neural networks on small datasets
 - Random forests with deep trees (without regularization)

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes bias (model assumptions)
- **C:** This describes irreducible error
- **D:** This describes a different type of error

EXAM TRAP: Variance = error from sensitivity to training data (overfitting)!

Q49: High Bias Characteristics - Tutorial

Which is a characteristic of a model with high bias?

ANSWER: B - Performs poorly on both training and test data (underfitting).

DETAILED EXPLANATION:

- **High bias pattern:**
 - Training error: HIGH
 - Test error: HIGH
 - Both errors are similar
 - Model is too simple
- **Why this happens:**
 - Model makes strong assumptions
 - Cannot capture underlying patterns
 - Underfits the data
- **Diagnosing high bias:**
 - Low training accuracy (for classification)
 - High training loss
 - No improvement with more data
 - Model predictions are overly smooth
- **Examples of high bias:**
 - Linear regression on quadratic data
 - Logistic regression with too few features
 - Shallow decision trees
 - Underparameterized neural networks
- **Fixing high bias:**
 - Increase model complexity
 - Add more features
 - Use more expressive model
 - Reduce regularization

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is low bias (fits training data well)

Q50: High Variance Characteristics - Tutorial

Which is a characteristic of a model with high variance?

ANSWER: B - Performs very well on training data but poorly on test data (overfitting).

DETAILED EXPLANATION:

- **High variance pattern:**
 - Training error: LOW
 - Test error: HIGH
 - Large gap between errors
 - Model is too complex
- **Why this happens:**
 - Model memorizes training data
 - Learns noise in addition to signal
 - Overfits to specific patterns
- **Diagnosing high variance:**
 - High training accuracy, low test accuracy
 - Model performance drops significantly on new data
 - Large coefficients (for linear models)
 - Complex decision boundaries
- **Examples of high variance:**
 - High-degree polynomial on small dataset
 - Deep neural networks with too many parameters
 - Decision trees with unlimited depth
 - Any model with too many parameters for data size
- **Fixing high variance:**
 - Add regularization (L1, L2, dropout)
 - Get more training data
 - Reduce model complexity
 - Use cross-validation

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is high bias (underfitting)

Q51: Underfitting Definition - Tutorial

What is underfitting in supervised learning?

ANSWER: B - Model is too simple to capture underlying patterns in data.

DETAILED EXPLANATION:

- **Definition:** Model fails to capture data's underlying structure
 - Too simple for the task
 - Cannot represent the true function
 - Makes overly strong assumptions
- **Signs of underfitting:**
 - High training error
 - High test error
 - No significant gap between training and test error
 - Predictions are too simplistic
- **Causes:**
 - Model is too simple (linear for nonlinear data)
 - Too few features
 - Too much regularization (too large λ)
 - Insufficient training (early stopping too early)
- **Fixing underfitting:**
 - Increase model complexity (more parameters)
 - Add more features
 - Decrease regularization
 - Train for more iterations
- **Analogy:** Trying to fit a sine wave with a straight line

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes overfitting
- **C:** This describes overfitting
- **D:** Too many parameters leads to overfitting

EXAM TRAP: Underfitting = model too simple = poor training and test performance!

Q52: Overfitting Definition - Tutorial

What is overfitting in supervised learning?

ANSWER: B - Model is too complex and learns the noise in the training data.

DETAILED EXPLANATION:

- **Definition:** Model fits training data too well, including noise
 - Memorizes training examples
 - Cannot generalize to new data
 - Captures random fluctuations
- **Signs of overfitting:**
 - Very low training error
 - High test error
 - Large gap between training and test error
 - Model has many parameters relative to data
- **Causes:**
 - Model is too complex
 - Too many features
 - Too little regularization (too small λ)
 - Too little training data
- **Fixing overfitting:**
 - Add regularization (L1, L2, dropout)
 - Get more training data
 - Reduce model complexity
 - Use cross-validation
 - Early stopping
- **Analogy:** Memorizing answers instead of learning concepts

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes underfitting
- **C:** This describes underfitting
- **D:** Too few parameters leads to underfitting

EXAM TRAP: Overfitting = model too complex = good training, poor test!

Q53: Causes of Overfitting - Tutorial

Which is a common cause of overfitting?

ANSWER: B - Too many parameters relative to number of training samples.

DETAILED EXPLANATION:

- **The core issue:** $p \gg N$ (more parameters than samples)
 - More parameters than constraints
 - Infinite solutions that fit data perfectly
 - Model can memorize individual examples
- **Example scenarios:**
 - 100 parameters with 50 samples
 - High-degree polynomial (degree 20) with 30 points
 - Deep neural network on small dataset
- **Other contributing factors:**
 - Too many features (high-dimensional)
 - Little or no regularization
 - Training for too many iterations
 - Data with high noise
- **Prevention strategies:**
 - Use cross-validation
 - Apply regularization
 - Reduce model complexity
 - Get more training data
 - Use feature selection

WHY OTHER OPTIONS ARE WRONG:

- **A:** Too few features leads to underfitting
- **C:** Too much regularization leads to underfitting
- **D:** Too simple a model leads to underfitting

EXAM TRAP: Overfitting happens when too many parameters for too few samples!

Q54: Causes of Underfitting - Tutorial

Which is a common cause of underfitting?

ANSWER: B - Insufficient model complexity (e.g., linear for nonlinear data).

DETAILED EXPLANATION:

- **The core issue:** Model cannot represent true function
 - Model class is too restrictive
 - Cannot capture the data's patterns
 - Relationship is more complex than model allows
- **Common scenarios:**
 - Linear regression for quadratic relationship
 - Logistic regression for complex decision boundary
 - Shallow network for deep patterns
 - Too few features
- **Other contributing factors:**
 - Too much regularization (λ too large)
 - Stopping training too early
 - Poor feature engineering
 - Too simple model architecture
- **Fixing underfitting:**
 - Choose more expressive model
 - Add more features
 - Decrease regularization
 - Train for more iterations
 - Use better feature engineering

WHY OTHER OPTIONS ARE WRONG:

- **A:** Too many parameters leads to overfitting
- **C:** Too little regularization leads to overfitting
- **D:** More training data generally helps

EXAM TRAP: Underfitting happens when model is too simple for the data!

Q55: Bias-Variance Tradeoff - Tutorial

As model complexity increases, what happens to bias and variance?

ANSWER: B - Bias decreases and variance increases.

DETAILED EXPLANATION:

- **Relationship:**
 - **Bias:** ↓ as complexity increases
 - **Variance:** ↑ as complexity increases
 - **Total error:** U-shaped curve
- **Why bias decreases:**
 - More flexible model captures more patterns
 - Can represent more complex functions
 - Assumptions are less restrictive
- **Why variance increases:**
 - Model becomes more sensitive to data
 - More parameters mean more degrees of freedom
 - Can fit noise as well as signal
- **The optimal point:**
 - Minimum of $\text{bias}^2 + \text{variance}$
 - Balance between underfitting and overfitting
 - Tradeoff is fundamental

WHY OTHER OPTIONS ARE WRONG:

- **A:** Both decrease is incorrect
- **C:** This is the opposite direction
- **D:** Both increase is incorrect

EXAM TRAP: More complexity → lower bias, higher variance!

Q56: Optimal Complexity - Tutorial

Where is the **optimal model complexity** located?

ANSWER: B - Where total error ($\text{bias}^2 + \text{variance} + \text{irreducible}$) is minimized.

DETAILED EXPLANATION:

• **Total error decomposition:**

- Total Error = $\text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$
- Bias^2 : decreases with complexity
- Variance: increases with complexity
- Irreducible: constant

• **Shape of error curve:**

- Low complexity: High bias, Low variance $\rightarrow$ High total error
- High complexity: Low bias, High variance $\rightarrow$ High total error
- Optimal: Sweet spot in between

• **Finding optimal complexity:**

- Use cross-validation
- Track validation error
- Choose model with lowest validation error

• **Practical considerations:**

- Optimal complexity depends on data size
- More data allows for more complex models
- Regularization helps shift the optimal point

WHY OTHER OPTIONS ARE WRONG:

- **A:** Highest variance = overfitting (too complex)
- **C:** Lowest bias = overfitting (too complex)
- **D:** Highest bias = underfitting (too simple)

EXAM TRAP: Optimal complexity = minimum total error ($\text{bias}^2 + \text{variance}$)!

Q57: Bias-Variance Example - Tutorial

Simple linear regression for nonlinear relationship would likely have:

ANSWER: A - High bias, low variance (underfitting).

DETAILED EXPLANATION:

- **Why high bias:**
 - Linear model assumes linear relationship
 - True relationship is nonlinear
 - Model cannot capture curvature
 - Systematically wrong predictions
- **Why low variance:**
 - Linear model is very simple
 - Few parameters (slope and intercept)
 - Not sensitive to training data
 - Stable predictions across datasets
- **Result:** Underfitting
 - High training error
 - High test error
 - Model is too simple
- **Fix:** Use polynomial regression or other nonlinear model

WHY OTHER OPTIONS ARE WRONG:

- **B:** This describes complex model (overfitting)
- **C:** Ideal but unrealistic for mismatched model
- **D:** Unlikely combination

EXAM TRAP: Too simple → high bias, low variance (underfitting)!

Q58: High-Degree Polynomial - Tutorial

High-degree polynomial on small dataset would likely have:

ANSWER: B - Low bias, high variance (overfitting).

DETAILED EXPLANATION:

- **Why low bias:**
 - High-degree polynomial is very flexible
 - Can fit any finite set of points
 - Captures all patterns in training data
 - Very expressive model
- **Why high variance:**
 - Many parameters (degree + 1)
 - Very sensitive to training data
 - Different dataset $\rightarrow$ very different fit
 - Oscillates between data points
- **Result:** Overfitting
 - Very low training error
 - High test error
 - Model is too complex
- **Fix:** Use regularization or simpler model

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes simple model (underfitting)
- **C:** Ideal but unrealistic for complex model on small dataset
- **D:** Unlikely combination

EXAM TRAP: Too complex $\rightarrow$ low bias, high variance (overfitting)!

Q59: Irreducible Error - Tutorial

What is irreducible error in the bias-variance decomposition?

ANSWER: C - Error due to noise in data that cannot be reduced by any model.

DETAILED EXPLANATION:

- **Definition:** Inherent noise in the data
 - Measurement errors
 - Unobserved variables
 - Inherent randomness
 - Natural variation
- **Mathematical notation:**
 - $Y = f(X) + \varepsilon$
 - $\varepsilon \sim \text{some distribution}$
 - $\text{Var}(\varepsilon) = \sigma^2 = \text{irreducible error}$
- **Why irreducible:**
 - No model can eliminate this error
 - Even if we knew true $f(X)$, still have ε
 - Best possible model still has error σ^2
- **Practical implications:**
 - There is always a limit to predictive accuracy
 - Cannot achieve zero error
 - Need to accept some level of error

WHY OTHER OPTIONS ARE WRONG:

- **A:** This describes bias (reducible error)
- **B:** This describes variance (reducible error)
- **D:** Loss function choice is a modeling decision

EXAM TRAP: Irreducible error = noise in data - cannot be eliminated!

Which statement about the bias-variance tradeoff is TRUE?

ANSWER: C - The optimal model balances bias and variance to minimize total error.

DETAILED EXPLANATION:

- **Key concept:** Balance is the goal
 - Not too simple (high bias)
 - Not too complex (high variance)
 - Find the sweet spot
- **Why other options are false:**
 - **A:** Simple models have high bias, low variance (opposite)
 - **B:** Complex models have low bias, high variance (opposite)
 - **D:** Bias and variance are related through complexity
- **How to balance:**
 - Use cross-validation to select model
 - Use regularization to control complexity
 - Monitor training and validation error
- **Real-world implications:**
 - No single best model for all problems
 - Tradeoff is fundamental
 - Choice depends on problem and data

WHY OTHER OPTIONS ARE WRONG:

- **A:** Simple models have high bias, low variance
- **B:** Complex models have low bias, high variance
- **D:** Bias and variance are related through model complexity

EXAM TRAP: Optimal model balances bias and variance!

Q61: Regularization Definition - Tutorial

What is regularization in supervised learning?

ANSWER: B - Technique to reduce overfitting by adding a penalty term to the loss function.

DETAILED EXPLANATION:

• **Definition:** Adding penalty to prevent overfitting

- $\tilde{L}(\theta) = L(\theta) + \lambda \cdot \text{Penalty}(\theta)$
- $L(\theta)$: original loss
- λ : regularization strength
- Penalty: encourages simpler models

• **Why it works:**

- Penalizes large parameters
- Reduces model complexity
- Improves generalization
- Controls the bias-variance tradeoff

• **Types of regularization:**

- L1 (Lasso): $\lambda \sum |w_i|$
- L2 (Ridge): $\lambda \sum w_i^2$
- Elastic Net: combination
- Dropout (neural networks)
- Early stopping

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is the opposite (increasing complexity)
- **C:** This is data augmentation
- **D:** This is feature selection (related but distinct)

EXAM TRAP: Regularization adds a penalty to the loss function!

Q62: L1 Regularization (Lasso) - Tutorial

What is L1 regularization (Lasso) and its key property?

ANSWER: B - Adds $\lambda \sum |w_i|$ to loss; can set weights to exactly zero (feature selection).

DETAILED EXPLANATION:

- **Lasso objective:**
 - $\tilde{L}(\theta) = L(\theta) + \lambda \sum_{j=1}^p |w_j|$
 - $L(\theta)$: original loss (e.g., MSE)
 - λ : regularization strength
- **Key property: Feature selection**
 - Can set weights to exactly zero
 - Effectively removes features
 - Sparse solutions
 - Identifies important features
- **Why it works:**
 - L1 penalty is non-differentiable at zero
 - Creates "corners" where weights become zero
 - Geometry: diamond-shaped constraint region
- **When to use:**
 - Many features, few important ones
 - Want interpretable model
 - Feature selection needed

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is L2 regularization (Ridge)
- **C:** Linear penalty is not standard
- **D:** Cubic penalty is not standard regularization

EXAM TRAP: L1 regularization can set weights to zero (feature selection)!

Q63: L2 Regularization (Ridge) - Tutorial

What is L2 regularization (Ridge) and its key property?

ANSWER: B - Adds $\lambda \sum w_j^2$ to loss; shrinks weights toward zero but does not set them to zero.

DETAILED EXPLANATION:

• **Ridge objective:**

- $\tilde{L}(\theta) = L(\theta) + \lambda \sum_{j=1}^p w_j^2$
- $L(\theta)$: original loss (e.g., MSE)
- λ : regularization strength

• **Key property: Weight shrinkage**

- Shrinks weights toward zero
- Never exactly zero (unless $\lambda = \infty$)
- Dense solutions
- All features retained

• **Why it works:**

- L2 penalty is differentiable everywhere
- Geometry: circular constraint region
- Smooth shrinkage

• **When to use:**

- Many features, all potentially important
- Multicollinearity present
- Want to keep all features
- Better predictive performance than L1 sometimes

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is L1 regularization (Lasso)
- **C:** Linear penalty is not standard
- **D:** Cubic penalty is not standard regularization

EXAM TRAP: L2 regularization shrinks weights but does not set them to zero!

Q64: Elastic Net - Tutorial

What is Elastic Net regularization?

ANSWER: A - Combination of L1 and L2: $\lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$

DETAILED EXPLANATION:

- **Elastic Net objective:**
 - $\tilde{L}(\theta) = L(\theta) + \lambda_1 \sum |w_i| + \lambda_2 \sum w_i^2$
 - Combines L1 and L2 penalties
 - Two hyperparameters: λ_1 and λ_2
- **Advantages:**
 - **Feature selection** (from L1)
 - **Group selection:** Correlated features enter together
 - **Better than Lasso** when features are correlated
 - **Better than Ridge** when only some features important
- **Why it's needed:**
 - Lasso struggles with correlated features (selects one randomly)
 - Ridge keeps all features
 - Elastic Net: correlation-aware feature selection
- **Common parameterization:**
 - α : L1 ratio ($\alpha = 1 = \text{Lasso}$, $\alpha = 0 = \text{Ridge}$)
 - λ : overall regularization strength
 - Penalty: $\lambda(\alpha \sum |w_i| + (1 - \alpha) \sum w_i^2)$

WHY OTHER OPTIONS ARE WRONG:

- **B:** This is Lasso (L1 only)
- **C:** This is Ridge (L2 only)
- **D:** This is no regularization

EXAM TRAP: Elastic Net = L1 + L2 regularization!

Q65: Regularization Parameter λ - Tutorial

What happens as the regularization parameter λ increases?

ANSWER: B - The model becomes simpler (weights shrink more toward zero).

DETAILED EXPLANATION:

- **Effect of increasing λ :**
 - Stronger penalty on large weights
 - Weights shrink closer to zero
 - Model becomes more constrained
 - Bias increases, variance decreases
- **Extreme cases:**
 - $\lambda = 0$: No regularization (standard OLS)
 - $\lambda \rightarrow \infty$: All weights $\rightarrow 0$ (underfitting)
- **Choosing λ :**
 - Use cross-validation
 - Balance between underfitting and overfitting
 - Typically choose λ that minimizes validation error
- **Model complexity relationship:**
 - λ controls model complexity
 - Larger $\lambda \rightarrow$ simpler model
 - Smaller $\lambda \rightarrow$ more complex model

WHY OTHER OPTIONS ARE WRONG:

- **A:** Larger λ means more regularization = simpler model
- **C:** Larger λ reduces overfitting
- **D:** λ definitely changes the model

EXAM TRAP: Larger $\lambda \rightarrow$ simpler model (more regularization)!

How does regularization prevent overfitting?

ANSWER: B - By penalizing large parameter values, reducing model complexity and sensitivity to training data.

DETAILED EXPLANATION:

• **Mechanism:**

- Large weights $\rightarrow$ complex decision boundaries
- Large weights $\rightarrow$ high sensitivity to inputs
- Penalty discourages large weights
- Result: smoother, simpler models

• **Why this prevents overfitting:**

- Model cannot fit noise (requires large weights)
- Less sensitive to training data fluctuations
- Better generalization to new data

• **Examples:**

- L2: $\lambda \sum w_i^2$ penalizes squared weights
- L1: $\lambda \sum |w_i|$ penalizes absolute weights
- Both discourage large weights

• **Analogy:** Occam's Razor - prefer simpler explanations

WHY OTHER OPTIONS ARE WRONG:

- **A:** Increasing features increases overfitting
- **C:** This is data augmentation
- **D:** More complex loss isn't regularization

EXAM TRAP: Regularization = penalizes large weights to reduce overfitting!

Q67: Dropout as Regularization - Tutorial

Is dropout a form of regularization? If so, how does it work?

ANSWER: B - Yes, randomly drops neurons during training, creating ensemble effect and preventing co-adaptation.

DETAILED EXPLANATION:

- **Definition:** Dropout is a regularization technique for neural networks
 - Randomly sets a fraction of neurons to zero
 - During each training iteration
 - Typical dropout rate: 0.2 to 0.5
- **How it works:**
 - **During training:** Each neuron has probability p of being dropped
 - **During testing:** All neurons active, scaled by $(1 - p)$
 - Prevents co-adaptation of neurons
- **Why it's effective:**
 - Creates an ensemble effect (many subnetworks)
 - Forces network to learn robust features
 - Prevents over-reliance on specific neurons
 - Acts as regularization (increases bias, decreases variance)
- **Practical notes:**
 - Used with large neural networks
 - Particularly effective in deep networks
 - Often combined with L2 regularization

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - dropout IS regularization
- **C:** False - dropout doesn't add penalty term
- **D:** False - dropout affects both forward and backward passes

EXAM TRAP: Dropout is a regularization technique for neural networks!

Q68: Early Stopping as Regularization - Tutorial

Is early stopping a form of regularization? If so, how does it work?

ANSWER: B - Yes, stops training before convergence, limiting model complexity and preventing overfitting.

DETAILED EXPLANATION:

- **Definition:** Early stopping stops training early
 - Monitor validation error
 - Stop when validation error starts increasing
 - Prevents overfitting
- **How it works:**
 - Track training and validation error
 - Validation error typically: decreases then increases
 - Stop at minimum validation error
 - Use parameters from best validation performance
- **Why it's regularization:**
 - Limits the number of training iterations
 - Equivalent to early stopping in optimization
 - Prevents fitting noise
 - Controls model complexity implicitly
- **Practical implementation:**
 - Patience parameter: wait for k epochs before stopping
 - Save best model checkpoint
 - Return to best checkpoint after stopping

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - early stopping IS regularization
- **C:** False - early stopping doesn't add penalty term
- **D:** False - early stopping affects model (prevents overfitting)

EXAM TRAP: Early stopping is a regularization technique!

Q69: Regularization Bias-Variance - Tutorial

How does regularization affect the bias-variance tradeoff?

ANSWER: C - Regularization increases bias and decreases variance.

DETAILED EXPLANATION:

- **Effect on bias:**
 - Regularization makes model simpler
 - Model may not fit training data perfectly
 - Systematic error (bias) increases
- **Effect on variance:**
 - Regularization constrains weights
 - Model is less sensitive to training data
 - Variability (variance) decreases
- **The tradeoff:**
 - Regularization shifts towards simpler models
 - Reduces overfitting (decreases variance)
 - Increases underfitting risk (increases bias)
 - Optimal λ balances the two
- **Visual representation:**
 - No regularization: low bias, high variance
 - Too much regularization: high bias, low variance
 - Optimal: balanced

WHY OTHER OPTIONS ARE WRONG:

- **A:** Variance decreases, not increases
- **B:** This is the opposite direction
- **D:** Regularization definitely affects both

EXAM TRAP: Regularization $\rightarrow$ increases bias, decreases variance!

Q70: When to Use Regularization - Tutorial

When should regularization be applied in supervised learning?

ANSWER: B - When the model is overfitting (high variance).

DETAILED EXPLANATION:

• **Indications for regularization:**

- Training error much lower than validation error
- Model performs well on training, poorly on test
- Large coefficients (for linear models)
- Complex model with limited data

• **Why it works:**

- Regularization reduces variance
- Prevents model from fitting noise
- Improves generalization

• **When NOT to use regularization:**

- Model is underfitting (high bias)
- Training error is already high
- Data is large enough for model complexity
- Model is interpretable and simple

• **Practical advice:**

- Always consider regularization for high-dimensional data
- Use cross-validation to select λ
- Start with no regularization, add if needed
- Balance bias and variance

WHY OTHER OPTIONS ARE WRONG:

- **A:** Underfitting would be worsened by regularization
- **C:** Too few parameters = underfitting
- **D:** Perfectly clean data still may benefit from regularization

EXAM TRAP: Use regularization when overfitting (high variance)!

Q71: L1 vs L2 Feature Selection - Tutorial

Which regularization method is more likely to perform feature selection?

ANSWER: B - L1 regularization (Lasso)

DETAILED EXPLANATION:

- **Why L1 performs feature selection:**
 - L1 penalty = $\lambda \sum |w_i|$
 - Non-differentiable at zero
 - Creates sparse solutions
 - Weights exactly zero $\Rightarrow$ features removed
- **Why L2 doesn't:**
 - L2 penalty = $\lambda \sum w_i^2$
 - Differentiable everywhere
 - Shrinks weights but never to zero
 - All features retained
- **Geometric intuition:**
 - L1 constraint: diamond shape
 - L2 constraint: circle shape
 - Diamond has corners on axes $\Rightarrow$ sparse solutions
- **Practical implications:**
 - Lasso for feature selection
 - Ridge for shrinkage
 - Elastic Net for both

WHY OTHER OPTIONS ARE WRONG:

- **A:** L2 only shrinks weights
- **C:** They are not equal
- **D:** L1 does perform feature selection

EXAM TRAP: L1 (Lasso) = feature selection; L2 (Ridge) = weight shrinkage!

Q72: Weight Zero Interpretation - Tutorial

In regularization, what does a weight of zero indicate?

ANSWER: B - The feature is not used by the model (feature selection).

DETAILED EXPLANATION:

• Weight interpretation:

- Weight w_j multiplies feature x_j
- $w_j = 0 \Rightarrow$ feature x_j contributes nothing
- Feature is effectively removed

• Why this happens:

- L1 regularization encourages sparsity
- Feature is not predictive enough
- Feature is redundant
- Regularization selects better features

• Benefit:

- Simpler model
- Better interpretability
- Reduced overfitting
- Faster prediction (fewer features)

• Caveat:

- Zero weights don't mean feature has no predictive power
- Feature may be correlated with others
- Feature may be less important than others

WHY OTHER OPTIONS ARE WRONG:

- **A:** Zero weight means no importance
- **C:** Weight doesn't indicate variance
- **D:** Weight doesn't indicate bias

EXAM TRAP: Weight = 0 $\rightarrow$ feature not used (feature selection)!

Q73: λ Too Large - Tutorial

If λ is too large in regularization, what happens?

ANSWER: B - The model underfits.

DETAILED EXPLANATION:

- **Effect of large λ :**
 - Penalty dominates the loss function
 - All weights shrink toward zero
 - Model becomes too simple
 - Cannot capture data patterns
- **Symptoms of underfitting:**
 - High training error
 - High test error
 - Model predictions are too simple
 - Weights all close to zero
- **Comparison:**
 - $\lambda = 0$: No regularization (may overfit)
 - λ small: Some regularization (balance)
 - λ large: Too much regularization (underfit)
 - $\lambda \rightarrow \infty$: All weights $\rightarrow 0$ (extreme underfit)
- **Fixing:**
 - Decrease λ
 - Use cross-validation to find optimal λ
 - Start with small λ and increase as needed

WHY OTHER OPTIONS ARE WRONG:

- **A:** Too small λ leads to overfitting
- **C:** Optimal requires balanced λ
- **D:** λ definitely affects model

EXAM TRAP: Too large $\lambda \rightarrow$ underfitting!

Q74: Regularization Forms - Tutorial

Which of the following is a form of regularization?

ANSWER: D - All of the above (L1, L2, Dropout)

DETAILED EXPLANATION:

- **L1 regularization (Lasso):**
 - Adds $\lambda \sum |w_i|$ to loss
 - Performs feature selection
 - Sparse solutions
- **L2 regularization (Ridge):**
 - Adds $\lambda \sum w_i^2$ to loss
 - Shrinks weights
 - Dense solutions
- **Dropout:**
 - Randomly drops neurons
 - Ensemble effect
 - Prevents co-adaptation
- **Other forms not listed:**
 - Early stopping
 - Batch normalization (has regularization effect)
 - Data augmentation
 - Weight decay (same as L2 in many frameworks)

WHY OTHER OPTIONS ARE WRONG:

- **A:** Correct but incomplete
- **B:** Correct but incomplete
- **C:** Correct but incomplete

EXAM TRAP: Many techniques are forms of regularization!

Q75: λ Purpose - Tutorial

What is the purpose of the regularization parameter λ ?

ANSWER: A - To control the tradeoff between fitting the training data and keeping the model simple.

DETAILED EXPLANATION:

- **Role of λ :**
 - Balances two competing objectives
 - Fit training data well (small λ)
 - Keep model simple (large λ)
 - Tradeoff controlled by λ
- **Extreme cases:**
 - $\lambda = 0$: Minimizes only training error
 - $\lambda \rightarrow \infty$: Minimizes only penalty (weights $\rightarrow 0$)
- **Choosing λ :**
 - Must be selected carefully
 - Use cross-validation
 - Tradeoff between bias and variance
 - Problem-specific
- **Relationship to other hyperparameters:**
 - Different from learning rate (α)
 - Different from number of epochs
 - Different from batch size

WHY OTHER OPTIONS ARE WRONG:

- **B:** Learning rate is α
- **C:** Number of epochs is different
- **D:** Batch size is different

EXAM TRAP: λ controls the bias-variance tradeoff in regularization!

Q76: Cross-Validation Definition - Tutorial

What is cross-validation in supervised learning?

ANSWER: B - Resampling technique used to evaluate model performance and select hyperparameters.

DETAILED EXPLANATION:

- **Definition:** Technique for assessing model generalization
 - Partitions data into training and validation sets
 - Multiple partitions for reliable estimate
 - Uses all data for both training and validation
- **Key purposes:**
 - **Model evaluation:** Estimate test performance
 - **Hyperparameter tuning:** Select best parameters
 - **Model comparison:** Choose between models
- **Why it's needed:**
 - Single train/test split is unreliable
 - Limited data: want to use all for training
 - Avoid overfitting to a specific split
- **Common methods:**
 - k-fold cross-validation
 - Leave-one-out cross-validation (LOOCV)
 - Stratified cross-validation

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is data augmentation
- **C:** This is regularization
- **D:** This is initialization

EXAM TRAP: Cross-validation evaluates model performance and selects hyperparameters!

Q77: K-Fold Cross-Validation - Tutorial

How does k-fold cross-validation work?

ANSWER: B - Split into k parts, train on $k-1$ parts and validate on remaining 1 part, repeat k times.

DETAILED EXPLANATION:

• **Step-by-step:**

- ① Randomly split data into k folds of equal size
- ② For $i = 1$ to k :
 - Use fold i as validation set
 - Use remaining $k - 1$ folds as training set
 - Train model and evaluate
- ③ Average performance over all k iterations

• **Example ($k=5$):**

- 1000 samples, 5 folds of 200 each
- Iteration 1: Train on 800, Validate on 200
- Repeat 5 times
- Average of 5 validation scores

• **Advantages:**

- All data used for training ($k-1$ times)
- All data used for validation (once)
- Reliable performance estimate
- Reduces overfitting to validation set

• **Common choices:**

- $k=5$, $k=10$ (most common)
- $k=3$ for very small datasets
- $k=10$ for moderate datasets

WHY OTHER OPTIONS ARE WRONG:

- **A:** Reverses training and validation
- **C:** All training = no validation
- **D:** All validation = no training

EXAM TRAP: K-fold CV = train on $k-1$ folds, validate on 1 fold, repeat k times!

Q78: Stratified Cross-Validation - Tutorial

What is stratified cross-validation?

ANSWER: B - Each fold preserves the same class distribution as the full dataset.

DETAILED EXPLANATION:

- **Definition:** CV with class balance preservation
 - Each fold has same proportion of each class
 - Matches the overall class distribution
 - Ensures representative validation sets
- **Example:**
 - Dataset: 90% class A, 10% class B
 - $k=5$ folds: each fold has 90% A, 10% B
 - Without stratification: some folds might have no B
- **When to use:**
 - Imbalanced datasets
 - Classification problems
 - When class proportions matter
- **Benefits:**
 - More reliable performance estimates
 - Avoids extreme validation sets
 - Better for rare classes
 - Reduces variance of estimates

WHY OTHER OPTIONS ARE WRONG:

- **A:** Equal fold sizes is standard k-fold
- **C:** Random folds may not preserve distribution
- **D:** Sequential folds may introduce bias

EXAM TRAP: Stratified CV preserves class proportions in each fold!

Q79: Stratified CV - When to Use - Tutorial

When is stratified cross-validation particularly important?

ANSWER: B - When the dataset is imbalanced (one class is much rarer than others).

DETAILED EXPLANATION:

- **Imbalanced dataset example:**
 - 95% negative class, 5% positive class
 - Without stratification: some folds may have 0% positive
 - Validation performance varies wildly
 - Estimates are unreliable
- **Why it's important:**
 - Rare class performance matters (fraud detection)
 - AUC, F1 need proper class representation
 - Avoids optimistic performance estimates
 - Better model selection
- **Without stratification:**
 - Fold may have no rare class samples
 - Cannot evaluate rare class performance
 - Model selection may choose wrong model
- **Practical advice:**
 - Always use stratification for classification
 - Especially when class imbalance exists
 - Also consider weighted metrics

WHY OTHER OPTIONS ARE WRONG:

- **A:** Balanced datasets don't need stratification
- **C:** Size doesn't determine need
- **D:** Stratification is about class distribution

EXAM TRAP: Stratified CV is essential for imbalanced datasets!

Q80: LOOCV Definition - Tutorial

What is Leave-One-Out Cross-Validation (LOOCV)?

ANSWER: A - Special case of k-fold CV where $k = N$ (one observation per fold).

DETAILED EXPLANATION:

- **Definition:** Extreme case of k-fold CV
 - $k = N$ where N is number of observations
 - Each fold contains exactly one observation
 - Train on $N - 1$ observations each time
- **Algorithm:**
 - 1 For $i = 1$ to N :
 - Hold out observation i as validation
 - Train on remaining $N - 1$ observations
 - Evaluate on held-out observation
 - 2 Average over all N iterations
- **Example:**
 - $N = 100$ observations
 - Train 100 models, each on 99 observations
 - Each validation set has 1 observation
 - Average validation error
- **Properties:**
 - Uses almost all data for training
 - Very low bias
 - High variance (estimates are correlated)
 - Computationally expensive

WHY OTHER OPTIONS ARE WRONG:

- **B:** $k=2$ is 2-fold CV
- **C:** $k=10$ is 10-fold CV
- **D:** LOOCV uses validation sets

EXAM TRAP: LOOCV = k-fold CV with $k = N$!

Q81: LOOCV Advantages - Tutorial

What is an advantage of Leave-One-Out Cross-Validation?

ANSWER: B - Uses almost all data for training each time ($N-1$ observations), giving low bias.

DETAILED EXPLANATION:

- **Low bias advantage:**
 - Training set size is $N - 1$ (almost full dataset)
 - Model sees almost all data each time
 - Training set is representative
 - No data withheld from training
- **Why bias is low:**
 - Model trained on nearly full dataset
 - No loss from using less data
 - Consistent with full data model
 - Less training-test discrepancy
- **Comparison to other CV:**
 - 10-fold CV: trains on 90% of data
 - 5-fold CV: trains on 80% of data
 - LOOCV: trains on 99%+ of data
 - LOOCV has lowest bias
- **When this matters:**
 - Small datasets (every observation counts)
 - When bias is a concern
 - When you want best estimate

WHY OTHER OPTIONS ARE WRONG:

- **A:** LOOCV is computationally expensive
- **C:** LOOCV can have high variance
- **D:** LOOCV is not suitable for large datasets

EXAM TRAP: LOOCV has low bias but is computationally expensive!

Q82: LOOCV Disadvantages - Tutorial

What is a disadvantage of Leave-One-Out Cross-Validation?

ANSWER: B - It is computationally expensive (requires N training runs).

DETAILED EXPLANATION:

- **Computational cost:**
 - Need to train model N times
 - Each training is on $N - 1$ observations
 - Total operations: $N \times \text{cost_per_train}$
 - For $N = 10,000$: 10,000 training runs
- **When it's problematic:**
 - Large datasets ($N > 1000$)
 - Complex models (slow to train)
 - Limited computational resources
- **Alternative solutions:**
 - Use k-fold with smaller k (e.g., 5 or 10)
 - Use approximate LOOCV formulas (for some models)
 - Use efficient implementations when available
- **Other disadvantages:**
 - High variance (estimates are correlated)
 - Can be unstable
 - Not suitable for large datasets

WHY OTHER OPTIONS ARE WRONG:

- **A:** LOOCV has low bias, not high bias
- **C:** LOOCV doesn't necessarily overfit
- **D:** LOOCV can be used for classification

EXAM TRAP: LOOCV is computationally expensive for large N !

Q83: CV Bias-Variance Tradeoff - Tutorial

How does choice of k in k -fold CV affect bias-variance tradeoff?

ANSWER: B - Larger k decreases bias and increases variance.

DETAILED EXPLANATION:

- **Effect of larger k :**

- More data used for training ($\frac{k-1}{k}$ fraction)
- Training sets more similar to full dataset
- Lower bias (models are better)
- Higher variance (estimates are more correlated)

- **Effect of smaller k :**

- Less data used for training
- Training sets less representative
- Higher bias (models are worse)
- Lower variance (estimates are less correlated)

- **The tradeoff:**

- LOOCV ($k=N$): Lowest bias, highest variance
- $k=2$: Highest bias, lowest variance
- $k=10$: Good balance (most common)

- **Choosing k :**

- $k=10$ is standard
- $k=5$ for larger datasets
- $k=N$ (LOOCV) for small datasets

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is the opposite direction
- **C:** k definitely affects bias-variance
- **D:** Smaller k increases bias, decreases variance

EXAM TRAP: Larger $k \rightarrow$ lower bias, higher variance!

Q84: Common k Value - Tutorial

What is the most common value of k in k -fold cross-validation?

ANSWER: C - $k = 10$

DETAILED EXPLANATION:

- **Why $k=10$ is common:**
 - Good balance between bias and variance
 - Computationally feasible
 - Widely studied and recommended
 - Standard in practice
- **Other common values:**
 - **$k=5$:** For larger datasets
 - **$k=10$:** Standard default
 - **$k=LOOCV$:** For small datasets
- **Rules of thumb:**
 - Use $k=10$ for most problems
 - Use $k=5$ for $N > 10,000$
 - Use LOOCV for $N < 100$
 - Increase k for smaller datasets
- **Tradeoffs for different k :**
 - $k=10$: 90% training, 10% validation
 - $k=5$: 80% training, 20% validation
 - $k=2$: 50% training, 50% validation (too little training)

WHY OTHER OPTIONS ARE WRONG:

- **A:** $k=2$ is sometimes used but not most common
- **B:** $k=5$ is also common but $k=10$ is most common
- **D:** LOOCV is used for small datasets

EXAM TRAP: $k = 10$ is the most common value!

Q85: CV Purpose - Tutorial

What is the primary purpose of cross-validation?

ANSWER: B - To evaluate how well a model will generalize to new, unseen data.

DETAILED EXPLANATION:

- **Primary goal:** Estimate generalization performance
 - How well does model perform on new data?
 - What is expected test error?
 - Can the model be trusted?
- **Why we need it:**
 - Training performance is optimistic
 - Test data is not available during development
 - Need reliable estimate without overfitting
- **Secondary purposes:**
 - Hyperparameter selection
 - Model comparison
 - Feature selection
 - Ensemble creation
- **What it doesn't do:**
 - Increase training data (that's data augmentation)
 - Reduce overfitting (that's regularization)
 - Select features (though it helps)

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is data augmentation
- **C:** This is feature selection
- **D:** This is a training parameter

EXAM TRAP: Cross-validation estimates generalization performance!

Q86: Bootstrapping Definition - Tutorial

What is bootstrapping in model evaluation?

ANSWER: A - Resampling technique that samples with replacement from the dataset.

DETAILED EXPLANATION:

- **Definition:** Sampling with replacement
 - Create B bootstrap samples
 - Each sample size = N (same as original)
 - Sampling with replacement
 - Some observations appear multiple times
 - Some observations don't appear
- **Expected composition:**
 - Each bootstrap sample has N observations
 - Approximately 63.2% of original data appears
 - Remaining 36.8% are out-of-bag (OOB)
- **Usage:**
 - Estimate variability of model performance
 - Construct confidence intervals
 - Evaluate stability
 - Used in bagging (Bootstrap Aggregating)
- **Example:**
 - Original: [1, 2, 3, 4, 5]
 - Bootstrap: [1, 3, 3, 4, 5] (3 appears twice)
 - Another: [1, 2, 2, 4, 4] (2 and 4 appear twice)

WHY OTHER OPTIONS ARE WRONG:

- **B:** Sampling without replacement is subsampling
- **C:** Bootstrapping is not regularization
- **D:** Bootstrapping is not feature selection

EXAM TRAP: Bootstrapping = sampling with replacement!

Q87: Bootstrapping Purpose - Tutorial

What is the primary purpose of bootstrapping in model evaluation?

ANSWER: A - To estimate the variability of model performance (e.g., confidence intervals).

DETAILED EXPLANATION:

- **Primary goal:** Quantify uncertainty
 - Model performance varies across samples
 - Need to know how much it varies
 - Construct confidence intervals
 - Assess model stability
- **How it works:**
 - Create B bootstrap samples
 - Train model on each sample
 - Evaluate performance on each sample (or OOB)
 - Get distribution of performance metrics
- **Applications:**
 - Confidence intervals for accuracy
 - Confidence intervals for coefficients
 - Model stability assessment
 - Parameter uncertainty quantification
- **Comparison to CV:**
 - CV: estimates average performance
 - Bootstrap: estimates performance distribution
 - Both useful for different purposes

WHY OTHER OPTIONS ARE WRONG:

- **B:** This is regularization
- **C:** Learning rate is a training parameter
- **D:** This is feature selection

EXAM TRAP: Bootstrapping estimates performance variability!

Q88: CV vs Bootstrapping - Tutorial

What is the key difference between cross-validation and bootstrapping?

ANSWER: A - Cross-validation samples without replacement; bootstrapping samples with replacement.

DETAILED EXPLANATION:

• Cross-validation:

- Samples without replacement
- Each observation appears exactly once in validation
- Partitions are disjoint
- Training sets are subsets of data
- No overlap between folds

• Bootstrapping:

- Samples with replacement
- Observations can appear multiple times
- Each sample is same size as original
- Some observations omitted (OOB)
- Overlap between samples

• Other differences:

- CV: deterministic (for given random seed)
- Bootstrap: random resampling
- CV: evaluates on held-out data
- Bootstrap: evaluates on sample or OOB

• When to use each:

- CV: model selection, hyperparameter tuning
- Bootstrap: uncertainty quantification, ensemble methods

WHY OTHER OPTIONS ARE WRONG:

- **B:** Both use parts of data
- **C:** Both can be used for either task
- **D:** They are different techniques

EXAM TRAP: CV = without replacement; Bootstrapping = with replacement!

Q89: Grid Search Definition - Tutorial

What is grid search in hyperparameter tuning?

ANSWER: B - Method that exhaustively searches over a predefined grid of hyperparameter values.

DETAILED EXPLANATION:

- **Definition:** Exhaustive parameter search
 - Define grid of values for each hyperparameter
 - Evaluate all combinations
 - Select best combination
 - Often combined with cross-validation
- **Example:**
 - λ : [0.001, 0.01, 0.1, 1]
 - Learning rate: [0.001, 0.01, 0.1]
 - Total combinations: $4 \times 3 = 12$
 - Each combination trained and evaluated
- **Advantages:**
 - Guarantees finding best in grid
 - Simple and easy to implement
 - Reproducible results
 - Parallelizable
- **Disadvantages:**
 - Computationally expensive
 - Curse of dimensionality
 - Continuous parameters difficult
 - May miss good values

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is random search
- **C:** This is gradient-based optimization
- **D:** Cross-validation is used WITH grid search

EXAM TRAP: Grid search = exhaustive search over parameter grid!

Q90: Grid Search Cost - Tutorial

What is a major disadvantage of grid search?

ANSWER: A - It is computationally expensive for large grids.

DETAILED EXPLANATION:

• Computational complexity:

- M hyperparameters, each with N_i values
- Total combinations: $\prod_{i=1}^M N_i$
- Grows exponentially with number of hyperparameters
- $M = 5$, $N = 10$ each $\Rightarrow$ 100,000 combinations

• When it's problematic:

- Many hyperparameters
- Large grids for each parameter
- Expensive models to train
- Combined with cross-validation (multiplies cost)

• Alternatives:

- Random search: sample random combinations
- Bayesian optimization: informed search
- Automated hyperparameter tuning
- Smaller grids with coarse-to-fine search

• Practical tips:

- Start with coarse grid
- Use random search for many parameters
- Use Bayesian optimization for expensive models

WHY OTHER OPTIONS ARE WRONG:

- **B:** Finds best within grid (not suboptimal)
- **C:** Can be used with CV
- **D:** Doesn't require gradients

EXAM TRAP: Grid search is computationally expensive for large grids!

Q91: Accuracy-Interpretability Tradeoff - Tutorial

What is the accuracy-interpretability tradeoff?

ANSWER: B - Tradeoff between predictive performance and how easily decisions can be understood.

DETAILED EXPLANATION:

- **Definition:** Tension between two competing goals
 - **Accuracy:** How well the model predicts
 - **Interpretability:** How well humans understand the model
 - Often in conflict
- **Why they conflict:**
 - Simpler models are more interpretable
 - Complex models are more accurate
 - Tradeoff is fundamental
 - Cannot maximize both
- **Examples:**
 - Linear regression: Very interpretable, limited accuracy
 - Random forest: Moderate interpretability, high accuracy
 - Deep neural network: Low interpretability, very high accuracy
- **Choosing the tradeoff:**
 - Depends on application
 - Regulated industries: prefer interpretability
 - Performance-critical: prefer accuracy

WHY OTHER OPTIONS ARE WRONG:

- **A:** This is speed-complexity tradeoff
- **C:** This is bias-variance tradeoff
- **D:** This is curse of dimensionality

EXAM TRAP: Accuracy vs Interpretability = performance vs understandability!

Q92: High Interpretability Models - Tutorial

Which model typically has high interpretability?

ANSWER: C - Decision trees (shallow)

DETAILED EXPLANATION:

- **Why decision trees are interpretable:**
 - Decision rules are easy to follow
 - Can be visualized
 - Each split corresponds to a rule
 - Path from root to leaf explains prediction
- **Example:**
 - If age ≥ 30 : predict high risk
 - If age ≤ 30 and income $\geq 50k$: predict low risk
 - Each prediction can be explained
- **Other interpretable models:**
 - Linear regression (coefficients explain)
 - Logistic regression (odds ratios)
 - Decision rules
 - Naive Bayes
- **Why others are less interpretable:**
 - DNNs: black-box, millions of parameters
 - Random forests: ensemble of many trees
 - GBM: many trees, complex interactions

WHY OTHER OPTIONS ARE WRONG:

- **A:** DNNs are black-box models
- **B:** Random forests are less interpretable
- **D:** GBM is complex ensemble

EXAM TRAP: Decision trees are highly interpretable "white-box" models!

Q93: High Accuracy Models - Tutorial

Which model typically has high accuracy but low interpretability?

ANSWER: C - Deep neural networks

DETAILED EXPLANATION:

- **Why DNNs are accurate:**
 - Can approximate any function
 - Learn hierarchical features
 - Massive parameter count
 - State-of-the-art on many tasks
- **Why DNNs are not interpretable:**
 - Millions of weights
 - No simple decision rules
 - Features are abstract
 - "Black-box" nature
 - Hard to explain individual predictions
- **Examples of high accuracy tasks:**
 - Image recognition (95%+ accuracy)
 - Speech recognition
 - Natural language processing
 - Game playing (AlphaGo)
- **Efforts to interpret DNNs:**
 - SHAP values
 - LIME
 - Saliency maps
 - Feature visualization

WHY OTHER OPTIONS ARE WRONG:

- **A:** Linear regression is highly interpretable
- **B:** Decision trees are highly interpretable
- **D:** Logistic regression is highly interpretable

EXAM TRAP: Deep neural networks = high accuracy, low interpretability!

Q94: Interpretability Importance - Tutorial

In which domain is interpretability particularly important?

ANSWER: C - Risk management and regulated industries (e.g., credit scoring)

DETAILED EXPLANATION:

- **Why regulated industries need interpretability:**
 - Legal requirements: must explain decisions
 - Regulatory compliance (GDPR, etc.)
 - Right to explanation for consumers
 - Avoiding discrimination
 - Auditing and accountability
- **Specific examples:**
 - **Credit scoring:** Must explain why loan denied
 - **Healthcare:** Must justify treatment decisions
 - **Insurance:** Must explain premium calculations
 - **Fraud detection:** Must justify alerts
- **Why other domains differ:**
 - Image recognition: interpretability less critical
 - NLP: accuracy often prioritized
 - Recommendations: personalization prioritized
- **Balance in risk management:**
 - White-box models preferred
 - Even if accuracy is lower
 - Explainability is legally required

WHY OTHER OPTIONS ARE WRONG:

- **A:** Image recognition prioritizes accuracy
- **B:** NLP often prioritizes accuracy
- **D:** Recommendations prioritize personalization

EXAM TRAP: Interpretability is critical in risk management and regulated industries!



Q95: Risk Management Model Choice - Tutorial

In risk management, which type of model is often preferred?

ANSWER: B - White-box models (e.g., decision trees, logistic regression)

DETAILED EXPLANATION:

- **Why white-box models are preferred:**
 - Regulatory requirements demand transparency
 - Must explain decisions to customers
 - Auditing is easier
 - Legal defensibility
 - Fairness can be verified
- **White-box examples in risk management:**
 - Logistic regression for credit scoring
 - Decision trees for fraud detection
 - Linear models for risk assessment
 - Scorecards in banking
- **Tradeoff accepted:**
 - May sacrifice some accuracy
 - Interpretability is worth the cost
 - Regulatory compliance is non-negotiable
- **When black-box models are acceptable:**
 - Internal research
 - Non-regulated applications
 - When explainability is not required

WHY OTHER OPTIONS ARE WRONG:

- **A:** Black-box models are avoided in risk management
- **C:** Ensembles are less interpretable
- **D:** Highest accuracy ignores interpretability requirements

EXAM TRAP: Risk management prefers interpretable (white-box) models!

Q96: Tradeoff Summary - Tutorial

Which statement about the accuracy-interpretability tradeoff is **TRUE**?

ANSWER: C - There is often a tradeoff between accuracy and interpretability.

DETAILED EXPLANATION:

- **Why there is a tradeoff:**
 - Interpretability requires simplicity
 - Accuracy often requires complexity
 - These goals are in tension
 - Cannot maximize both simultaneously
- **Examples of the tradeoff:**
 - Linear regression: Interpretable but limited
 - Random forest: Less interpretable but more accurate
 - DNN: Hard to interpret but highly accurate
- **When the tradeoff is less severe:**
 - Simple problems: both can be achieved
 - Domain knowledge helps interpretability
 - Some techniques improve both
- **Managing the tradeoff:**
 - Choose based on application
 - Use interpretability tools (SHAP, LIME)
 - Consider hybrid approaches

WHY OTHER OPTIONS ARE WRONG:

- **A:** False - simpler models often have lower accuracy
- **B:** False - complex models often have higher accuracy
- **D:** False - they are related through complexity

EXAM TRAP: Accuracy and interpretability are often in tension (tradeoff)!

Q97: Feature Importance - Tutorial

What is feature importance in model interpretability?

ANSWER: A - Measure of how important each feature is in making predictions.

DETAILED EXPLANATION:

- **Definition:** Quantifies feature contribution
 - Which features drive predictions?
 - How much does each feature contribute?
 - Helps understand model behavior
 - Identifies most important predictors
- **Common methods:**
 - Coefficient magnitude (linear models)
 - Feature importance (tree-based models)
 - Permutation importance
 - SHAP values
 - LIME weights
- **Interpreting feature importance:**
 - Higher value = more important
 - Zero = no importance
 - Positive vs negative direction matters
 - Context-dependent
- **Applications:**
 - Feature selection
 - Model explanation
 - Domain insight
 - Debugging models

WHY OTHER OPTIONS ARE WRONG:

- **B:** This is data availability
- **C:** This is feature variance
- **D:** This is correlation

EXAM TRAP: Feature importance helps interpret model predictions!

Q98: SHAP Values - Tutorial

What are SHAP (SHapley Additive exPlanations) values?

ANSWER: A - Method to compute feature importance using game theory.

DETAILED EXPLANATION:

- **Definition:** SHapley Additive exPlanations
 - Based on Shapley values from game theory
 - Distributes prediction among features
 - Each feature gets a contribution
 - Unifies other explanation methods
- **Key properties:**
 - **Additivity:** Sum of SHAP values = prediction
 - **Consistency:** If model changes, values change accordingly
 - **Local accuracy:** Explains individual predictions
 - **Missingness:** Missing features get SHAP=0
- **Advantages:**
 - Globally and locally consistent
 - Works for any model
 - Provides both direction and magnitude
 - Interpretable and reliable
- **Example interpretation:**
 - Base prediction: 0.5
 - Feature A contribution: +0.3
 - Feature B contribution: -0.1
 - Final prediction: 0.7

WHY OTHER OPTIONS ARE WRONG:

- **B:** SHAP is for explanation, not accuracy
- **C:** SHAP is not regularization
- **D:** SHAP is not hyperparameter tuning

EXAM TRAP: SHAP uses game theory to explain predictions!

What is LIME (Local Interpretable Model-agnostic Explanations)?

ANSWER: A - Method to explain individual predictions by approximating model locally with interpretable model.

DETAILED EXPLANATION:

- **Definition:** Local Interpretable Model-agnostic Explanations
 - Explains individual predictions
 - Approximates complex model locally
 - Uses interpretable model (linear, decision tree)
 - Model-agnostic (works for any model)
- **How it works:**
 - 1 Select instance to explain
 - 2 Generate perturbed samples around it
 - 3 Get predictions from complex model
 - 4 Fit interpretable model on perturbed data
 - 5 Use interpretable model to explain prediction
- **Key features:**
 - **Local:** Only accurate near the instance
 - **Model-agnostic:** Works with any model
 - **Interpretable:** Uses simple model
 - **Sparse:** Uses few features
- **Example:**
 - Complex DNN predicted loan denial
 - LIME says: low income and high debt caused denial
 - Explanation is local to that case

WHY OTHER OPTIONS ARE WRONG:

- **B:** LIME is for explanation, not accuracy
- **C:** LIME is not regularization
- **D:** LIME is not feature selection (though it identifies important features locally)

EXAM TRAP: LIME provides local explanations for individual predictions!

Q100: Model Estimation Summary - Tutorial

Which provides the most comprehensive summary of model estimation?

ANSWER: B - Model estimation involves choosing between OLS, NLS, and MLE, optimizing using gradient descent, and evaluating with cross-validation.

DETAILED EXPLANATION:

• Complete picture of model estimation:

① Estimation method selection:

- OLS: linear in parameters
- NLS: nonlinear in parameters
- MLE: maximum likelihood

② Optimization:

- Gradient descent (and variants)
- Backpropagation (neural networks)
- Numerical methods for NLS

③ Evaluation:

- Cross-validation
- Bootstrap
- Performance metrics

④ Regularization:

- L1/L2 regularization
- Dropout
- Early stopping

• Key insight: All components work together

- Each step is important
- Choices affect final model
- Tradeoffs at every step

WHY OTHER OPTIONS ARE WRONG:

- **A:** Only OLS is incomplete
- **C:** Only MLE is incomplete
- **D:** Only regularization is incomplete

EXAM TRAP: Model estimation is comprehensive - includes method choice, optimization, and evaluation!

Summary: Complete Review

Core Estimation Methods:

- **OLS:** Minimizes SSE; closed-form for linear regression; BLUE under Gauss-Markov
- **MLE:** Maximizes likelihood; equivalent to OLS under normal errors
- **NLS:** For non-linear-in-parameters models; requires numerical optimization

Optimization:

- **Gradient Descent:** Iterative; uses negative gradient; learning rate controls step size
- **Backpropagation:** Chain rule for neural networks; requires storing activations
- **Adam/Momentum:** Adaptive methods for faster convergence

Model Selection:

- **Bias-Variance Tradeoff:** Simplicity vs complexity; optimal at minimum total error
- **Regularization:** L1 (Lasso - feature selection), L2 (Ridge - shrinkage)
- **Cross-Validation:** k-fold (k=10 common); evaluates generalization

Interpretability:

- **White-box models:** Decision trees, linear regression (interpretable)
- **Black-box models:** Deep neural networks (accurate but not interpretable)
- **Explainability:** SHAP values, LIME for understanding predictions

Exam Success: Understand the tradeoffs and know the definitions!